

基本知识

高林

中国科学院计算技术研究所

提纲

■ 重要概念

- ◆ 颜色视觉
- ◆ 图像和像素
- ◆ 三角网格模型
- ◆ 光照模型与明暗处理
- ◆ 视点变换和视点方向

■ 绘制管线

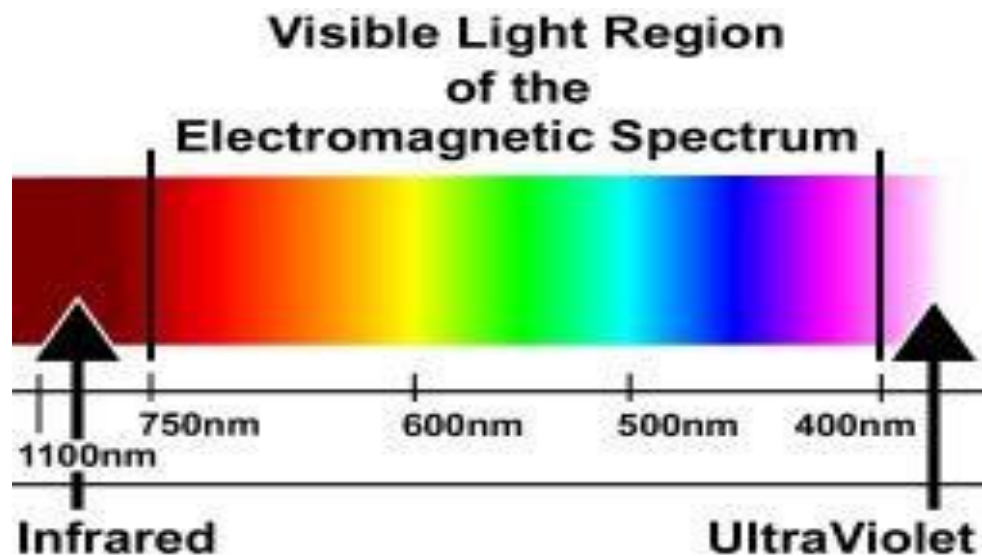
Part 1: 重要概念

1. 颜色视觉
2. 图像和像素
3. 三角网格模型
4. 光照模型与明暗处理
5. 视点变换和视点方向

颜色视觉

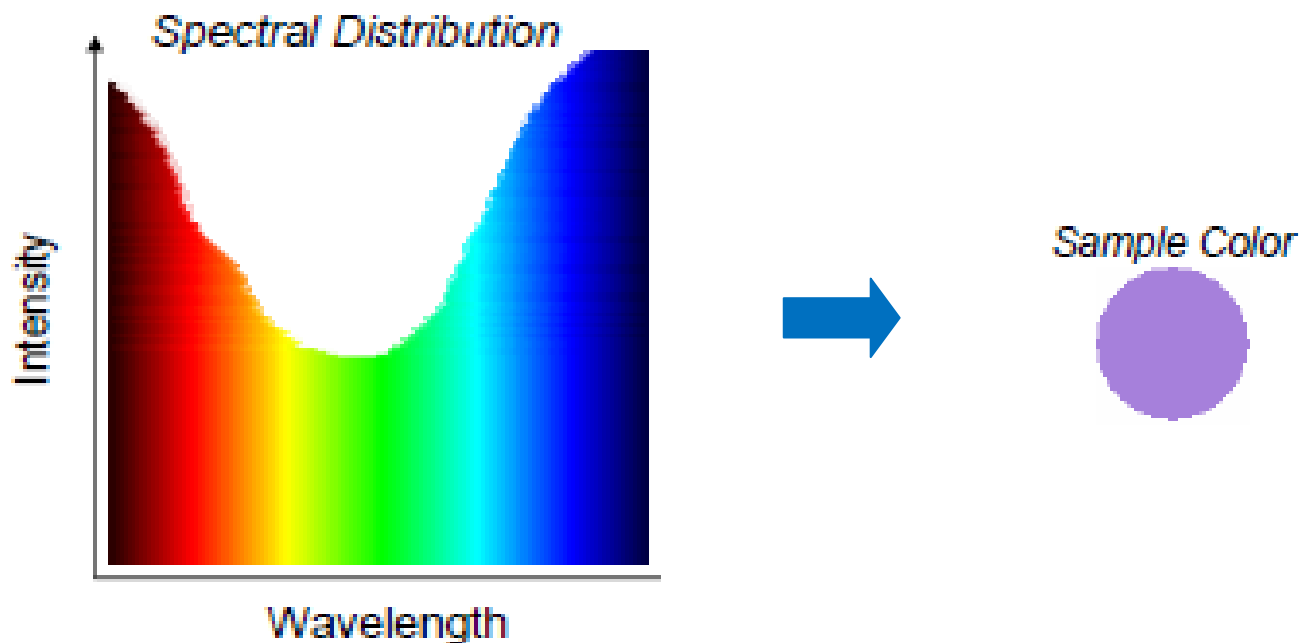
■ 什么是颜色?

- ◆ 颜色是对不同波长的光的能量的感知：不同波长的电磁波对应不同的颜色；对于人眼能感知的光（可见光），其波长范围为 380nm 到 760 nm 之间。



光的谱分布

- 光是由不同波长的电磁波按照某种能量分布混合叠加而成。
- 例如，白光是由所有可见波长的电磁波以相等的强度混合得到。
- 谱分布：光在各个可见波长的分量的强度分布函数称为光的谱分布。



颜色vs光的谱分布

- 与光类似，颜色也可以使用谱分布函数来进行描述。
- 然而，使用谱分布函数来表示颜色，不仅复杂，而且这样的表示方法并不是一个一一对应关系。
- 实际上，不同的谱分布函数可能对应同一种颜色，也就是异谱同色现象。

RGB 颜色空间

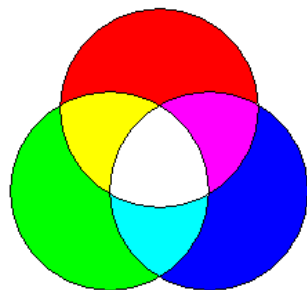
- 在所有用于表示色彩的各种颜色空间中，RGB（红绿蓝）颜色空间在计算机图形学中的使用最为广泛：
 - ◆ 颜色使用三通道 RGB 向量 (r,g,b) 来表示；
 - ◆ 在 RGB 颜色空间中，常用操作通常是对 RGB 三通道分别处理；
 - ◆ 通常可以将 r,g,b 分别规整化为 $[0,1]$ 内的浮点数；当使用 8bit 进行存储时， r,g,b 通常取值为 $[0,255]$ 内的整数。

RGB 颜色空间

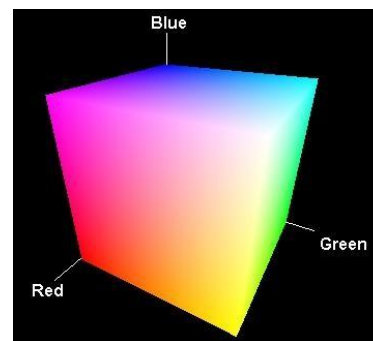
- 颜色表示为三种基本颜色：红色 (R), 绿色 (G), 蓝色 (B) 的线性组合

$$\mathbf{C} = r\mathbf{R} + g\mathbf{G} + b\mathbf{B}$$

- 为什么选择红绿蓝作为基本颜色？
- 以人类视觉的三刺激理论为基础，人眼的视网膜中有三种锥状视觉细胞，分别对红、绿、蓝三种光最敏感。

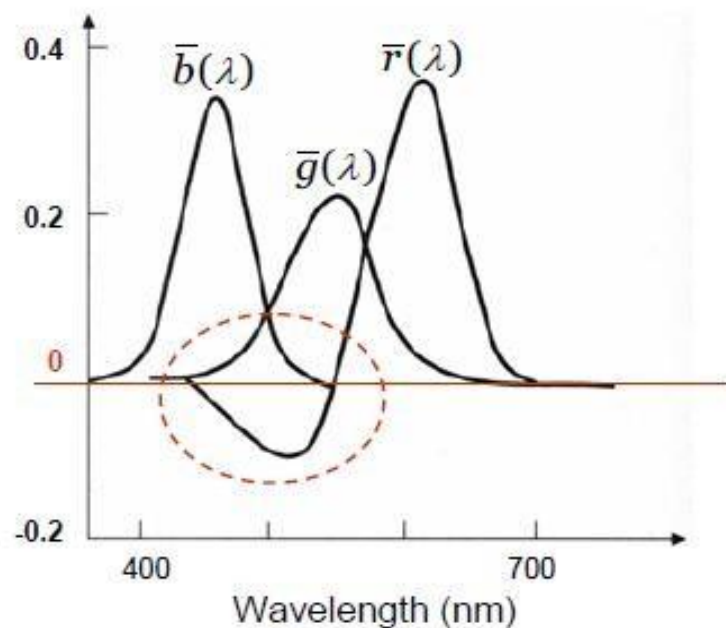


RGB 颜色空间的图示



RGB 颜色空间

- 然而，一部分色彩无法表示成 R, G, B 光波的正线性组合，这是 RGB 颜色空间的一个缺点。
- 右图为所有可见光波的 RGB 分量图：



其他颜色空间

- 其他常用的颜色空间还包括：

- ◆ CMY
- ◆ HSV
- ◆ CIE XYZ

CMY 颜色空间

- CMY：不同于 RGB 的另外一组基本颜色。
- Cyan (青), Magenta(品红), Yellow (黄)：分别是 R, G, B 的补色 (complements)。
- CMY 被称为 “减色系统”：
- RGB 为 “加色系统”：(0, 0, 0)为黑, (1, 1, 1)为白；
- CMY：(0, 0, 0)为白, (1, 1, 1)为黑。

$$\begin{array}{|c|} \hline \text{C} \\ \hline \end{array} = \begin{array}{|c|} \hline 1 \\ \hline \end{array} - \begin{array}{|c|} \hline \text{R} \\ \hline \end{array}$$
$$\begin{array}{|c|} \hline \text{M} \\ \hline \end{array} = \begin{array}{|c|} \hline 1 \\ \hline \end{array} - \begin{array}{|c|} \hline \text{G} \\ \hline \end{array}$$
$$\begin{array}{|c|} \hline \text{Y} \\ \hline \end{array} = \begin{array}{|c|} \hline 1 \\ \hline \end{array} - \begin{array}{|c|} \hline \text{B} \\ \hline \end{array}$$

HSV 颜色空间

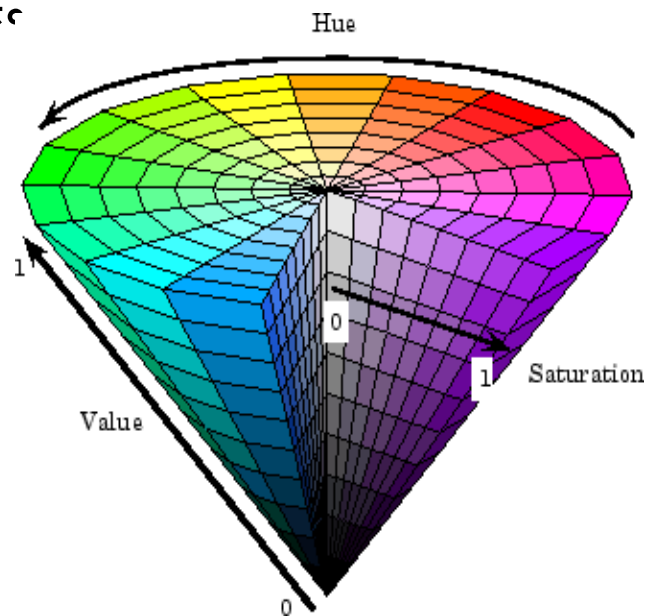
- 在真实的 RGB 颜色图像中，可以包含多达 $256 \times 256 \times 256 = 16,777,216$ 种可能的颜色。使用 RGB 颜色空间来描述和定位如此大量的不同颜色非常困难。
- HSV 系统则提供了一个直观的方法来对颜色进行准确的选择。
- HSV颜色空间应用于：艺术领域，以及图像处理、分形图像...

HSV 颜色空间

■ HSV：圆锥形的颜色空间

- ◆ Hue (色调) 也叫色相，它描述了色彩的本征属性，即我们常说的一个色彩是 红、橙、黄、绿、青、蓝、紫，等等。
- ◆ Saturation (饱和度) 也叫纯度.饱和度越低，色彩越白。
- ◆ Value of brightness (亮度)：亮度越低，色彩越黑。

■ HSV 比 RGB 更加用户友好。



CIE XYZ 颜色空间

■ CIE XYZ 颜色空间

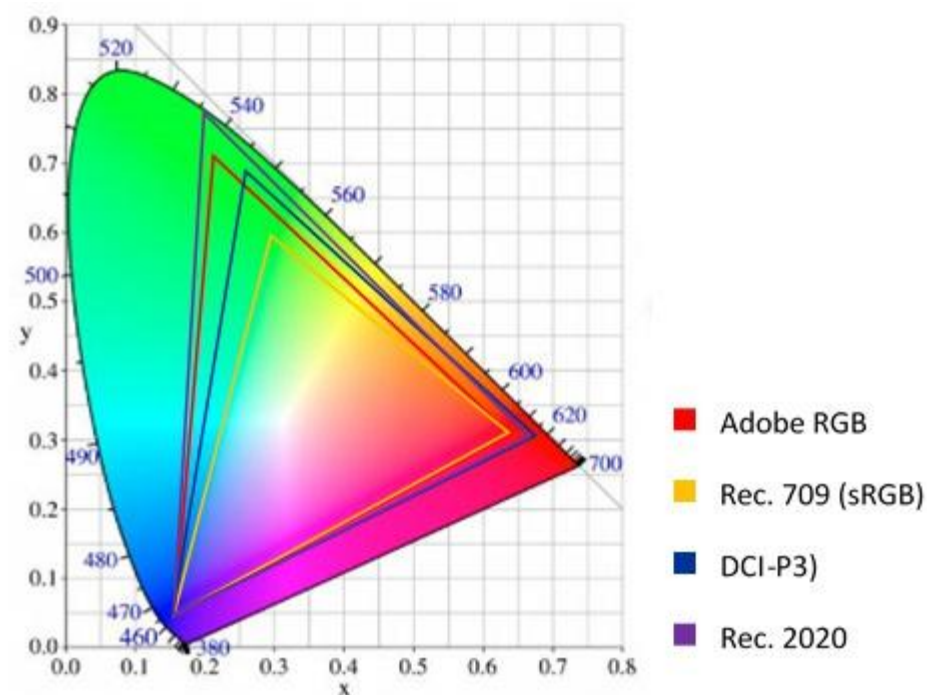
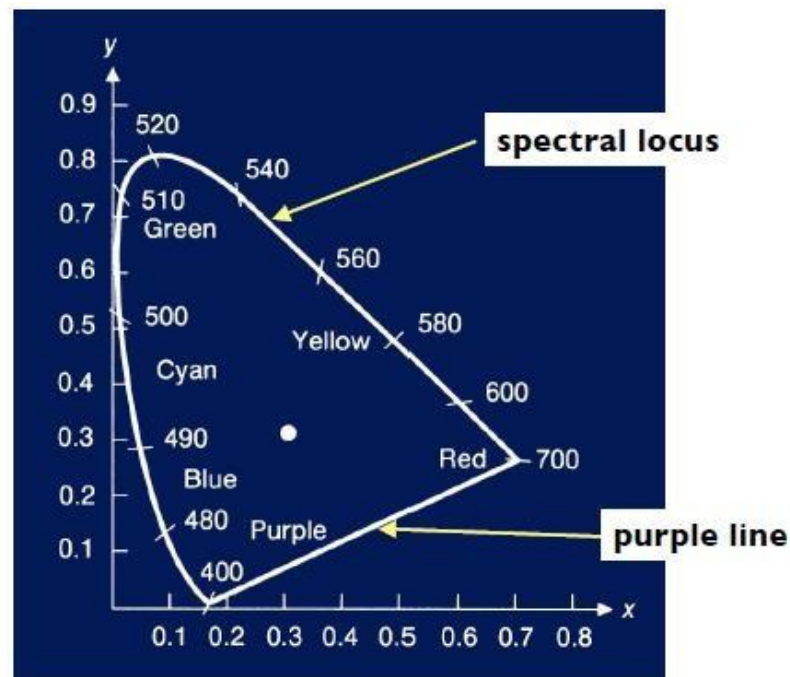
- ◆ 由国际发光照明委员会于1931年提出。
- ◆ CIE XYZ 色彩空间可以表示所有可感知的颜色 (而 RGB 空间却不能)。
- ◆ 更多地应用于颜色科学的研究。
- ◆ 与人类的感知系统密切相关。

CIE XYZ 色度图

■ 可以将 CIE XYZ 颜色空间可视化如下色度图，其中：

◆ $x = X/(X + Y + Z)$

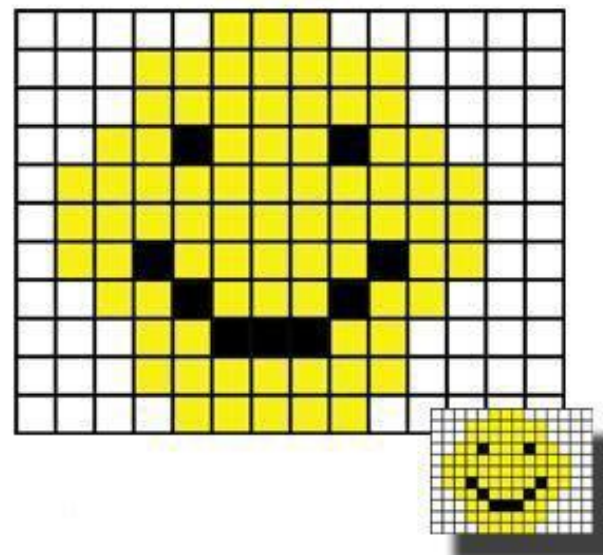
◆ $y = Y/(X + Y + Z)$



图像和像素

■ 图像

- ◆ 图像可以看成是一个二维离散函数： $f(x,y)$
- ◆ 函数 f 的定义域是由矩阵排列着的许多格子组成，这些格子被称为像素 (pixel)。
 - ◆ 函数 f 的取值则为各个像素 的色彩：
 - 对于彩色图像，可以是 RGB 或者 RGBA；
 - 对于灰度图像， f 为单值函数。



三角网格模型

- 图形学的基本目标是什么？
- 从虚拟的三维场景及相机的位置信息中，生成出一幅二维图像。
- 而三维场景又以怎样的数据结构来表示？
 - ◆ 简单的球体，长方体可直接用其参数描述。
 - ◆ 对于复杂模型，则需要使用参数曲线和曲面或者更一般的网格模型来进行描述。网格模型之中又以三角网格最为常用。

三角网格的定义

- 三角网格是由一系列欧氏空间中的三维顶点以及连接这些顶点的若干三角面片组成，具体包括：
 - ◆ 顶点集合 $V = (v_1, v_2, \dots, v_n)$
 - ◆ 面片集合 $F = (f_1, f_2, \dots, f_m)$
- 其中 F 中的每个面片 f_i 都是由 V 中的顶点构成的空间三角形：
 - ◆ $f_1 = (v_{a1}, v_{b1}, v_{c1}), f_2 = (v_{a2}, v_{b2}, v_{c2}), \dots$

几个例子

- 模型“牛”上显示了三角网格的结构。
- 模型“龙”和“人头”则是利用三角网格进行绘制的结果。



法向量

■ 三角面片的法向量 (normal):

- ◆ 三角面片的法向量是垂直于该三角面片所在平面的非零向量;
- ◆ 对于每个三角面片单独而言, 其法向量都有两种可能的朝向;
- ◆ 法向量的朝向决定了一个三角面片的正面与反面;
- ◆ 对于连续可定向的三角网格整体而言, 相邻的三角面片需要具备一致的法向量朝向。

法向量

- 三角网格顶点的法向量可以通过其周围的所有三角面片的法向量通过加权叠加计算：
- 假设 V 是 k 个三角面片 $f_1, f_2, \dots, f_k$ 所共有的顶点；
- 按算术平均计算：

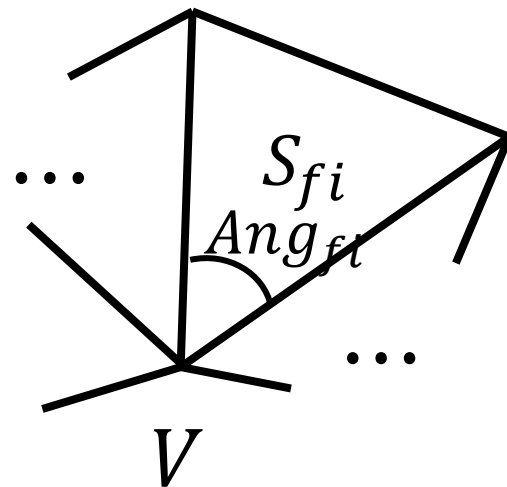
$$N_v = (N_{f_1} + \dots + N_{f_k}) / k$$

- 按面积加权平均计算：

$$N_v = (S_{f_1}N_{f_1} + \dots + S_{f_k}N_{f_k}) / (S_{f_1} + \dots + S_{f_k})$$

- 按角度加权平均计算：

$$N_v = (Ang_{f_1}N_{f_1} + \dots + Ang_{f_k}N_{f_k}) / (Ang_{f_1} + \dots + Ang_{f_k})$$

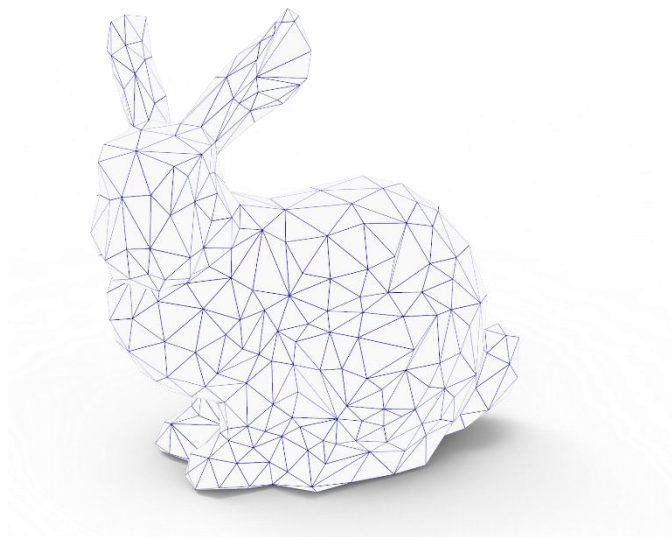


三角网格的简单绘制

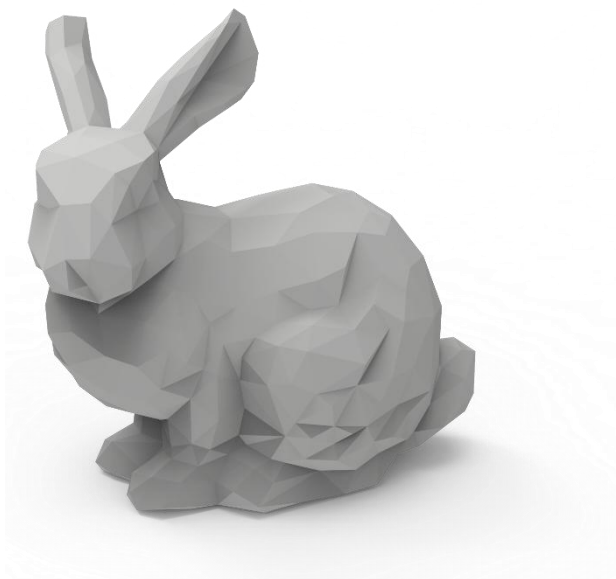
- 三角网格模型的每个顶点需要指定一个颜色属性。
- 基于颜色的绘制：
 - ◆ 模型表面的每点的颜色通过其所在三角面片的顶点颜色插值得到。
- 基于光照的绘制：
 - ◆ 需要指定一个虚拟的光照环境；
 - ◆ 如何计算光照对颜色的影响是最大的问题。

三角网格的简单绘制

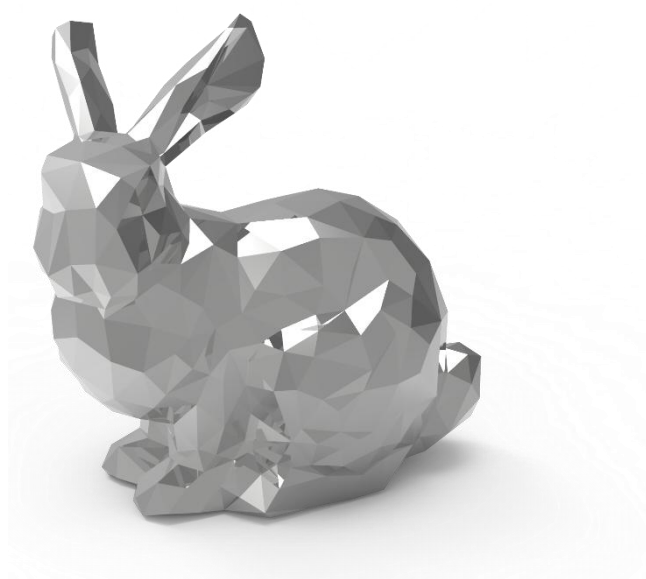
线框效果



颜色效果



光照效果



光照模型

- 光照模型 (lighting model 或 illumination model) 用于计算光的强度:
- 局部光照明 (Local Lighting)
 - ◆ 关注物体直接受到光源影响所产生的光照效果。
- 全局光照明 (Global Lighting)
 - ◆ 关注阴影效果;
 - ◆ 关注所有不是直接与光源位置相关的光照效果, 例如反射和折射效果, 等等
 - ...

光照模型的历史

- 1967年, Wylie 等人第一次在显示物体时加入了光照效果, 认为光的强度与物体到光源的距离成反比关系。
- 1970年, Bouknight 提出了第一个光反射模型:
- Lambert 漫反射光 + 环境光, 发表于 Communication of ACM

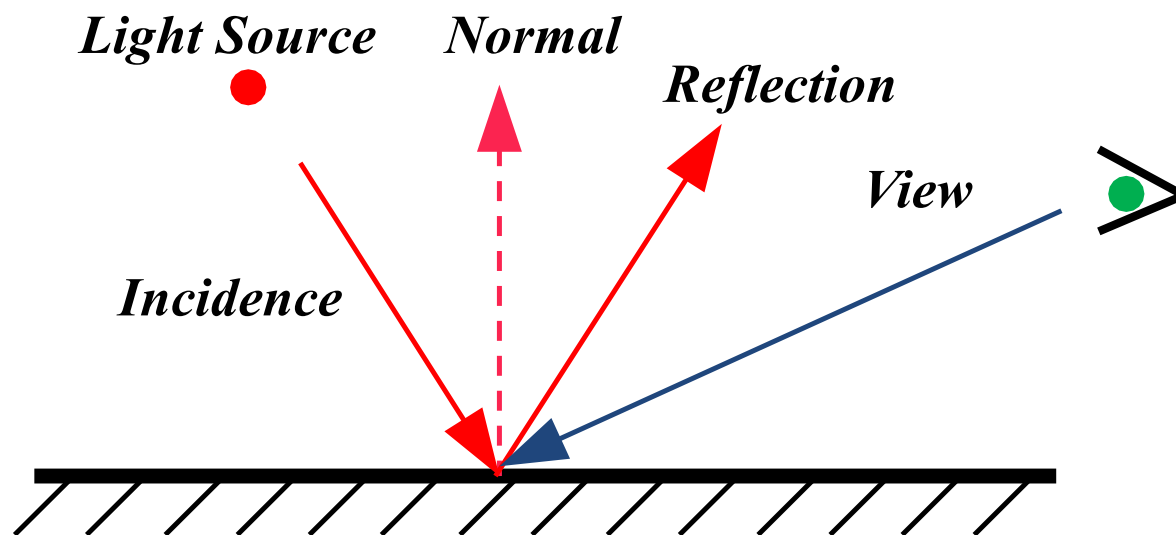
光照模型的历史

- 1971年, Gouraud 提出了漫反射模型加插值的思想:
 - ◆ Lambert 漫反射光 + Barycentric 插值
 - ◆ 发表于 IEEE transactions on Computers
- 1975年, Phong 提出了图形学中第一个有影响也是最有影响的光照模型: Phong 模型
 - ◆ 漫反射 (diffuse light) + 环境光 (ambient light) + 高光(specular light)
 - ◆ 发表于 Communication of ACM

光的传播

■ 光的传播遵循反射定律：

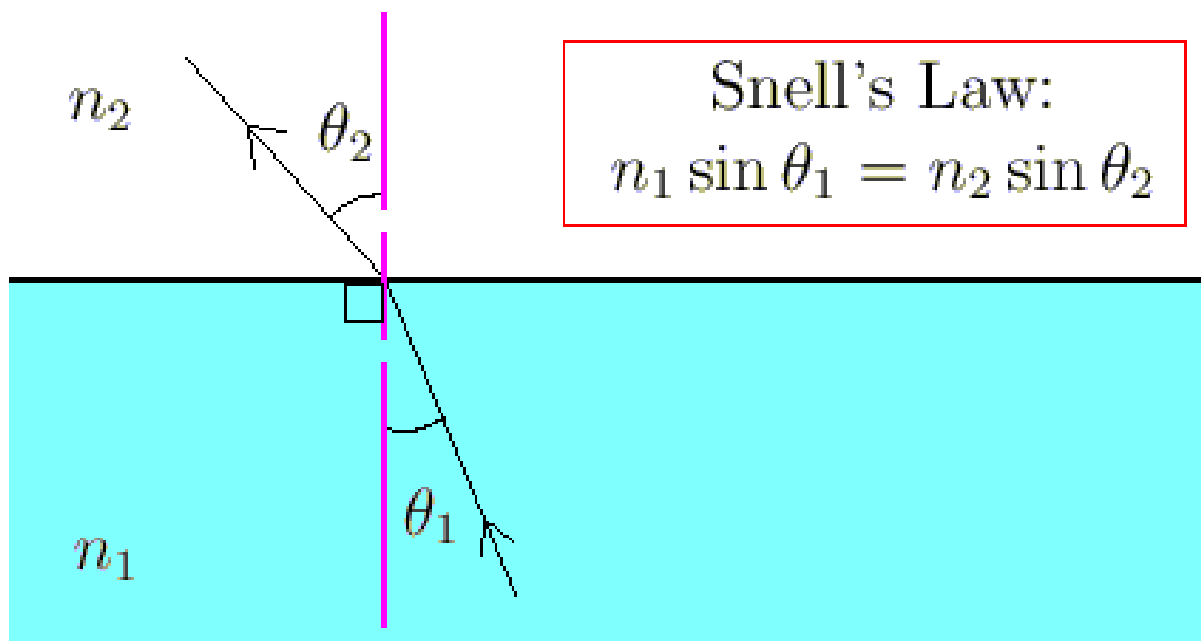
- ◆ 入射角等于反射角；
- ◆ 入射光线、反射光线、以及反射面的法向量位于同一平面内。



光的传播

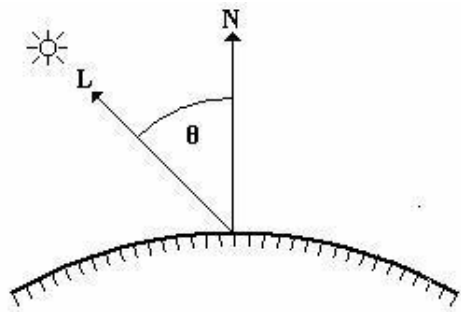
■ 折射定律 (也叫 Snell 定律) :

- ◆ 入射角和折射角的正弦值之比是一个仅仅取决于介质的常数;
- ◆ 这个常数称为相对折射系数。



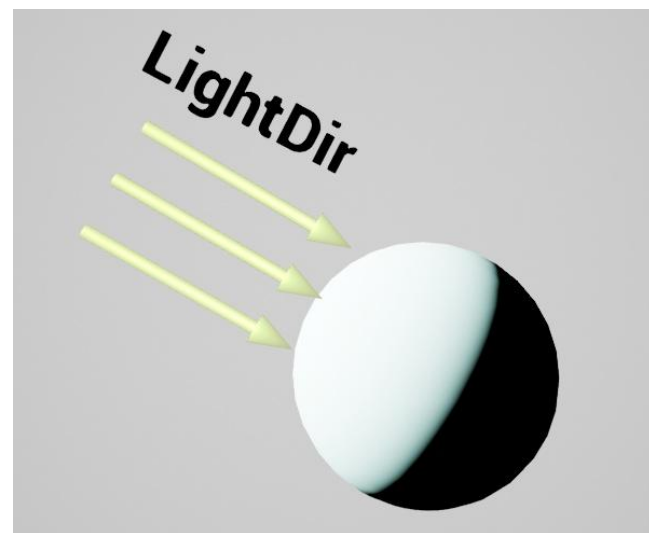
Lambert光照模型

- Lambert光照模型主要是用来模拟粗糙物体表面的光照现象。



$$I_d = K_d * I * \text{dot}(N, L)$$

- K_d 是漫反射系数。
- K_d 具有三个分量 k_{dr}, k_{dg}, k_{db} 分别代表 R, G, B三个通道的漫反射系数。
- K_d 与模型自身的色彩紧密相关。

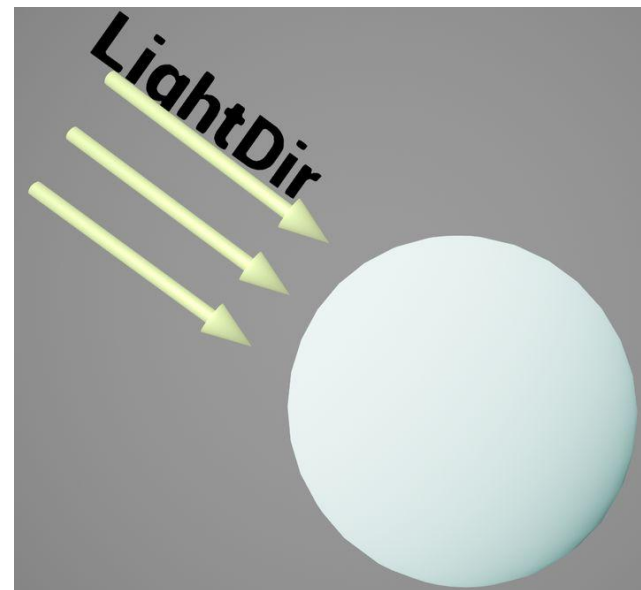


Half Lambert光照模型

- Half Lambert光照模型正式对Lambert模型的计算结果进行了重新映射，将[0,1]的计算结果置换到了[0.5,1]之间，目的是为了提升物体整体亮度。

$$I_d = K_d * I * (\text{dot}(N, L) * 0.5 + 0.5)$$

- K_d 是漫反射系数。
- K_d 具有三个分量 k_{dr}, k_{dg}, k_{db} 分别代表R, G, B三个通道的漫反射系数。
- K_d 与模型自身的色彩紧密相关。



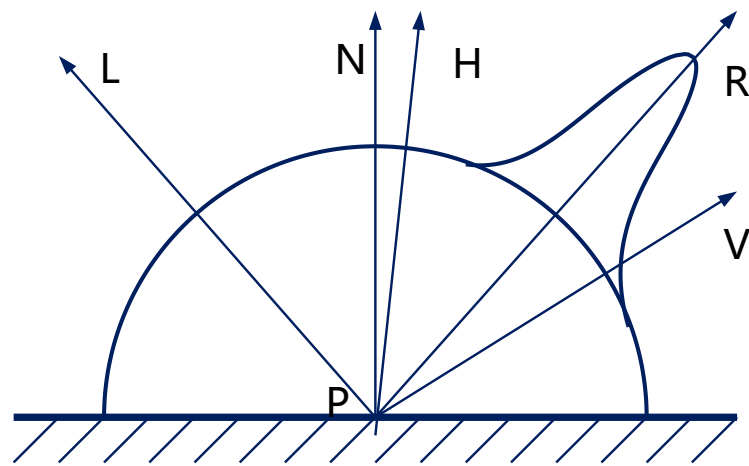
Phong 光照模型

- Phong 模型支持点光源和方向光源。
- Phong 模型是局部光照模型，将局部光照明效果分解为三个部分：
 - ◆ 漫反射光效果
 - ◆ 镜面反射光效果
 - ◆ 环境光效果

Phong 光照模型

- Phong 模型支持点光源和方向光源。
- Phong 模型是局部光照模型，将局部光照明效果分解为三个部分：
 - ◆ 漫反射光效果
 - ◆ 镜面反射光效果
 - ◆ 环境光效果

L 是入射光；R 是反射光；N 是物体表面的法向量；V 是视点方向；H 是 L 和 V 夹角的角平分线方向。



Phong 光照模型

- Phong 模型支持点光源和方向光源。
- Phong 模型是局部光照模型，将局部光照明效果分解为三个部分：
 - ◆ 漫反射光效果
 - ◆ 镜面反射光效果
 - ◆ 环境光效果

Phong 光照模型

- 漫反射光效果
- 漫反射光的传播是各向同性的;
- 漫反射光的强度为:

$$I_d = I_i K_d * (L \cdot N)$$

- K_d 是漫反射系数。
- K_d 具有三个分量 k_{dr}, k_{dg}, k_{db} 分别代表 R, G, B三个通道的漫反射系数。
- K_d 与模型自身的色彩紧密相关。

Phong 光照模型

- 镜面反射光效果
- 对于光滑的平面，依据反射定律，反射光线往往集中在一个小的立体角内，这些反射光我们称之为镜面反射光；
- 镜面反射光的强度为：

$$I_s = I_i K_s * (R \cdot V)^n$$

- K_s 是镜面反射系数，与物体表面光滑程度相关；
- n 是反射指数； n 越大，则高光区域越集中。

Phong 光照模型

- 环境光效果
- 环境光的强度为：

$$I_a = I_i K_a$$

- K_a 是物体对环境光的反射系数。
- 视角方向的发光强度为漫反射光分量、镜面反射光分量，以及环境光分量的发光强度之和：

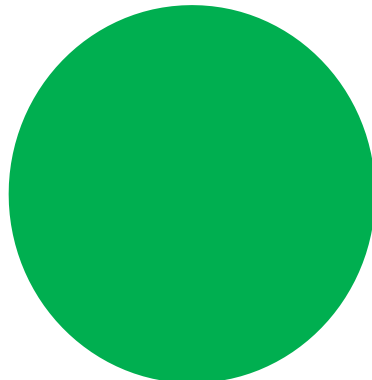
$$I = I_i K_a + I_i K_s * (R \cdot V)^n + I_i K_d * (L \cdot N)$$

Phong 光照模型



漫反射光

+



环境光

+

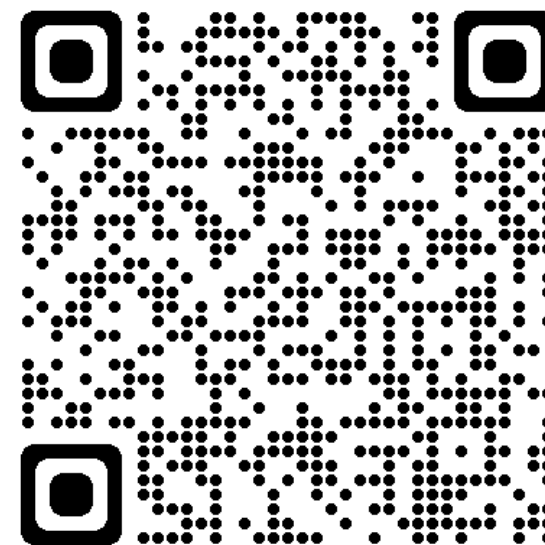
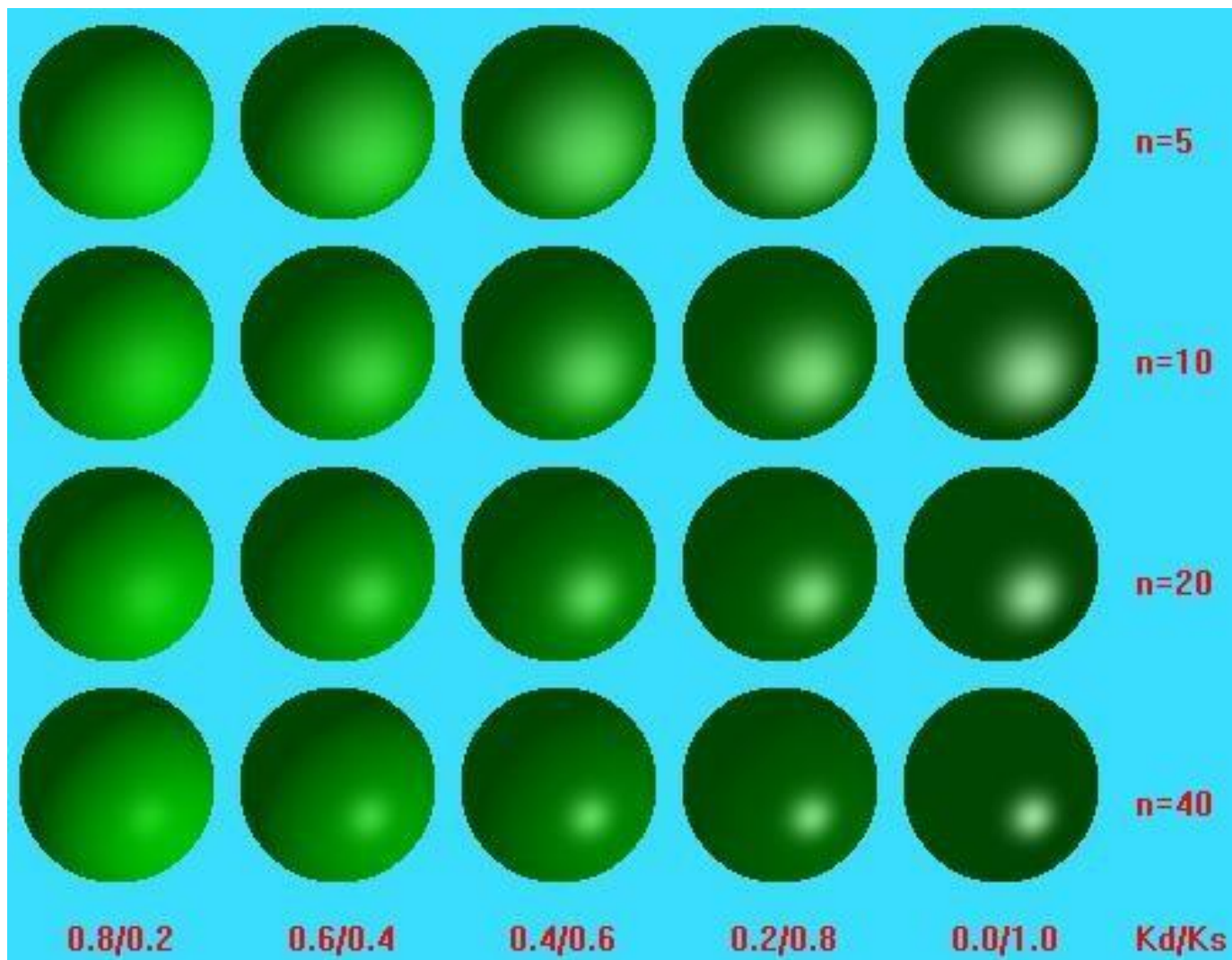


镜面反射光

=



Phong 模型示例



交互示例二维码

<http://multivis.net/lecture/phong.html>

明暗处理 (Shading)

- 考虑到物体表面的几何细节往往并不规则，为了减缓由模型离散化所导致的不光滑的色彩效果，通常的明暗处理除了使用光照模型外，还需要进行插值。
- Gouraud 明暗处理和Phong 明暗处理
 - ◆ Gouraud 明暗处理是对色彩进行插值；
 - ◆ Phong 明暗处理则是对法向进行插值。

Gouraud 明暗处理

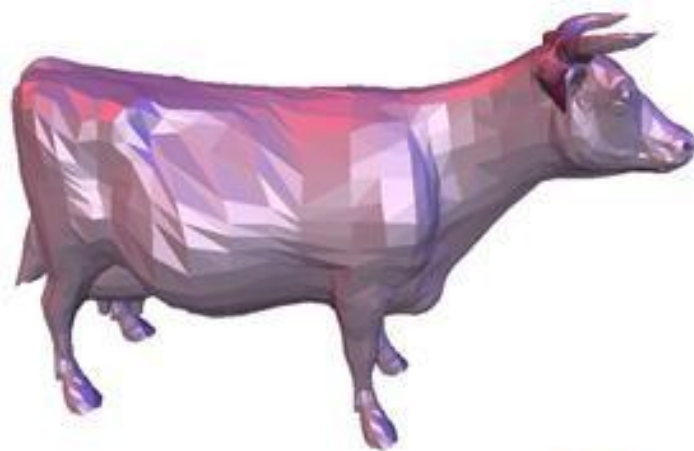
- 由 Gouraud 于 1971 年提出
 - ◆ 又被称为 Gouraud 插值。
- 计算方法：
 - ◆ 计算多边形的顶点法向量
 - ◆ 用光照模型去计算每个顶点的光强
 - ◆ 用双线性插值计算多边形表面上每个像素的明暗

Phong 明暗处理

- 由 Bui Tuong Phong 于 1973 年在他的博士论文中提出，与 Phong 光照模型是不同的概念。Phong 明暗处理与 Gouraud 明暗处理类似，区别在于进行双线性插值的不是光照强度本身，而是顶点的法线。因此使用这种着色法计算出的高光比 Gouraud 着色更精确。
- 计算方法：
 - ◆ 计算多边形顶点的法向量
 - ◆ 双线性插值计算每个像素点的法向量
 - ◆ 通过每个像素的法向量计算光强
 - ◆ 根据光强绘制像素

结果示例

直接用光照明模型
进行绘制



Phong 明暗处理



视点变换和视点方向

- 图形学关注于如何将由几何模型组成的三维场景绘制成高质量的彩色图像。
- 变换在图形学中至关重要：通过变换,可以简洁高效地设置和编辑三维场景，光照位置，以及视点方向。

视点变换和视点方向

- 图形学关注于如何将由几何模型组成的三维场景绘制成高质量的彩色图像。
- 变换在图形学中至关重要：通过变换,可以简洁高效地设置和编辑三维场景，光照位置，以及视点方向。假设我们已经有了了一段可以绘制正方形 $[0,1] \times [0,1]$ 的代码。如果现在我们需要绘制另外一个一条对角线是从 (lox, loy) 到 (hix, hiy) 的矩形，我们应该怎么做？

视点变换和视点方向

- 一种方法是写一段新的代码：

```
drawRect(lox, loy, hix, hiy) {  
    glBegin(GL_QUADS);  
    glVertex2f(lox, loy);  
    glVertex2f(hix, loy);  
    glVertex2f(hix, hiy);  
    glVertex2f(lox, hiy);  
    glEnd();  
}
```

- 矩形简单可以这么做，可是对于复杂的例子（例如绘制一个茶壶），怎么办？

视点变换和视点方向

```
drawRect(lox, loy, hix, hiy) {  
    glTranslate(lox, loy);  
    glScale(hix-lox, hiy-loy);  
    drawUnitSquare(0,0,1,1);  
}
```

- 这样的基于变换的解决方案在图形学中随时都需要用到；使用变换可以使代码更加快速、灵活、模块化。

视点变换和视点方向

- 变换是一个将空间中的点 x 映射成其他点 x' 的函数。
- 广泛应用于：Morphing, Deformation, Viewing, Projection, Real-time shadows...

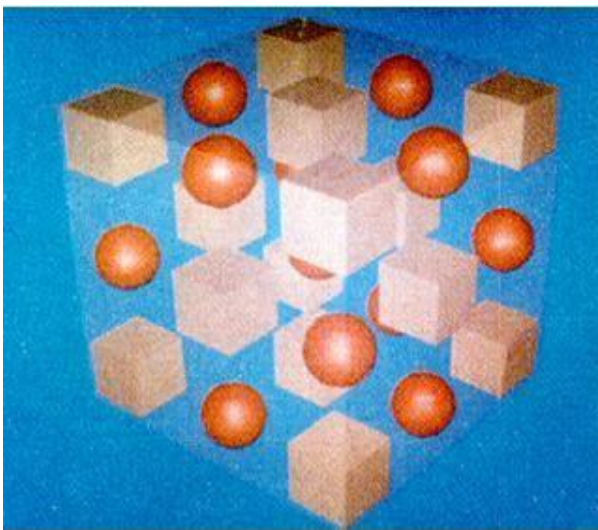
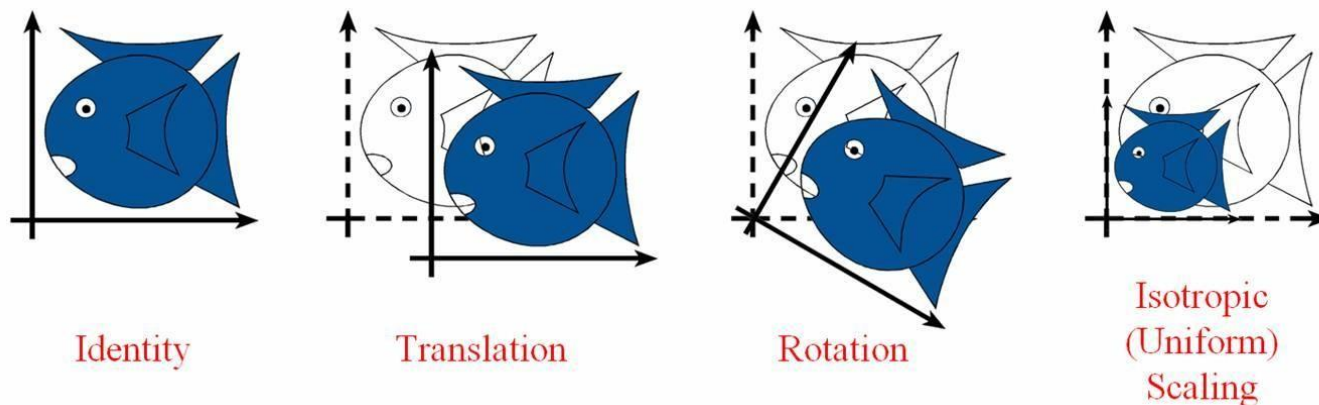


Fig 1. Undeformed Plastic



Fig 2. Deformed Plastic

视点变换和视点方向



- 从左到右的变换分别是：

不变 (Identity), 平移 (Translation), 旋转 (Rotation), 均衡缩放 (Isotropic scaling)

- 变换可以相互复合和嵌套：

例如：先旋转，再缩放，最后再平移。

- 简单变换都是可逆的。

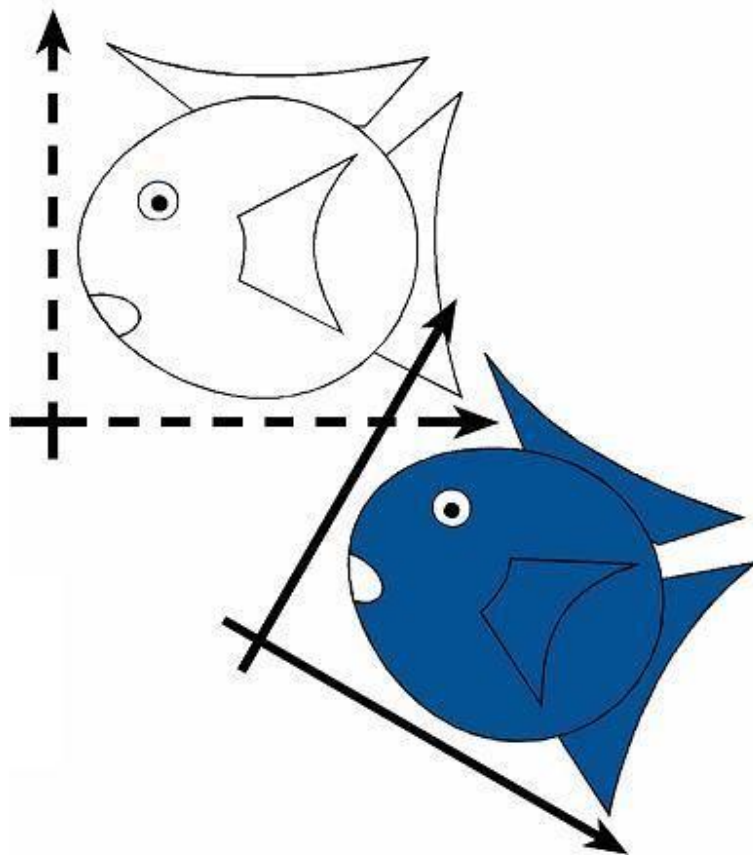
视点变换和视点方向

■ 常见的变换有如下几类：

- ◆ 刚体变换 (Rigid-body Transformation)
- ◆ 相似变换 (Similarity Transformation)
- ◆ 线性变换 (Linear Transformation)
- ◆ 仿射变换 (Affine Transformation)
- ◆ 投影变换 (Projective Transformation)

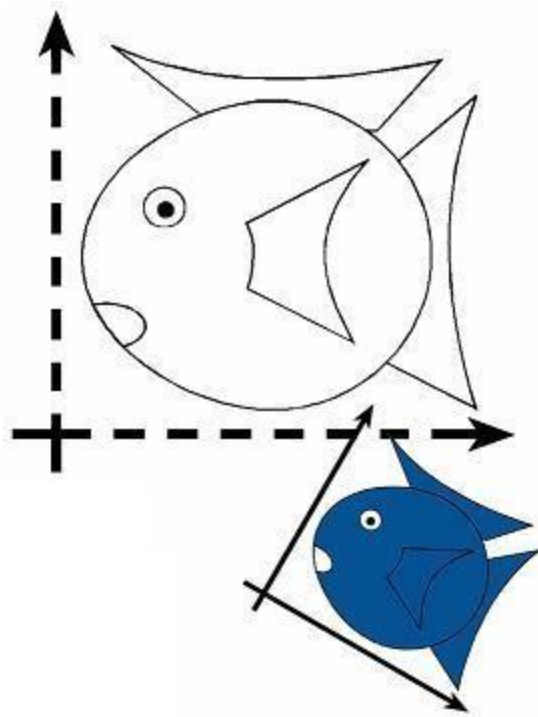
刚体变换

- 保持度量（长度、角度、大小）
- 刚体变换包括：
 - ◆ 不变
 - ◆ 平移
 - ◆ 旋转
 - ◆ 以及它们的复合



相似变换

- 保持角度
- 相似变换包括：
 - ◆ 不变
 - ◆ 平移
 - ◆ 旋转
 - ◆ 均衡缩放
 - ◆ 以及它们的复合



线性变换

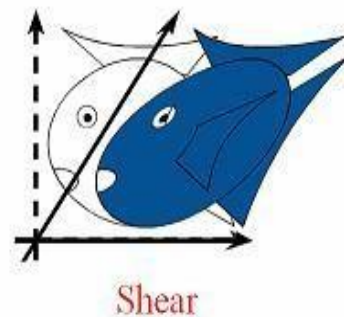
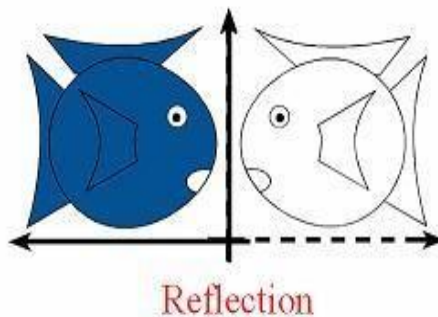
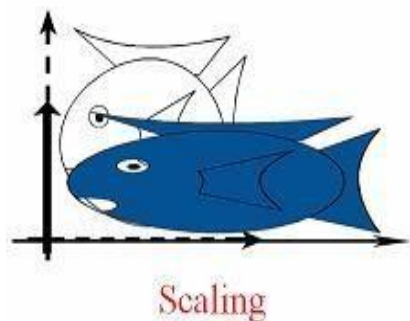
- 线性变换满足如下方程：

$$L(p+q) = L(p) + L(q)$$

$$aL(p) = L(ap)$$

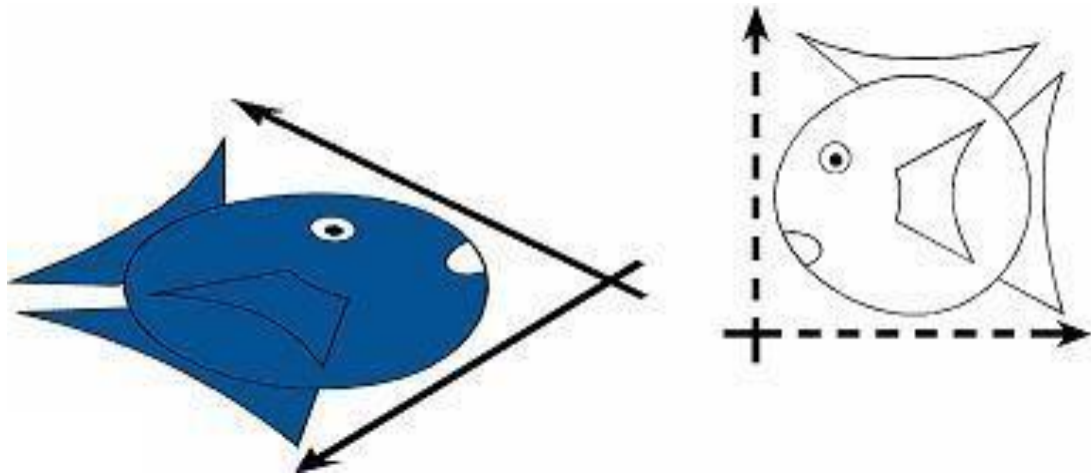
- 线性变换包括：

- ◆ 不变、旋转、缩放 (不一定要均衡缩放)
- ◆ 对称 (Reflection), 错切 (Shear)



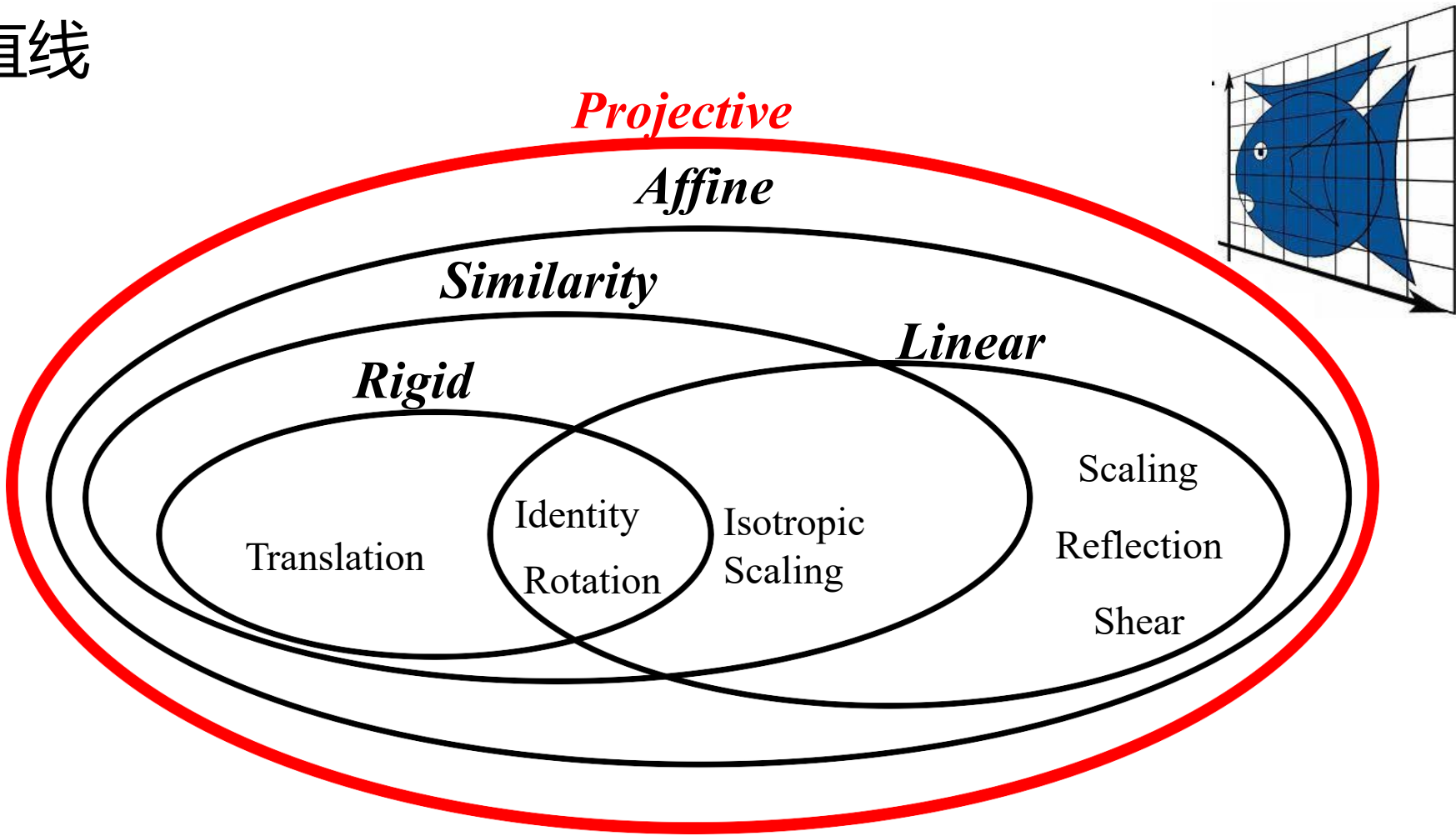
仿射变换

- 保持直线以及直线与直线的平行。
 - ◆ 两条在仿射变换之前平行的直线，在仿射变换之后依旧平行。
- 仿射变换包括：
 - ◆ 线性变换
 - ◆ 相似变换
 - ◆ 以及它们的复合



投影变换

- 保持直线



变换的表示

- 考虑简单的二维仿射变换:

$$x' = ax + by + c$$

$$y' = dx + ey + f$$

- 可以将两式统一地写成矩阵形式:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ d & e \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} c \\ f \end{bmatrix}$$

$$p' = Mp + t$$

变换的表示

- 变换的表示涉及到了两个变量： M 和 t

$$p' = Mp + t$$

- 然而，使用齐次坐标 (Homogenous Coordinates) 来表示 p 和 p' ，则上述公式成为：

$$p' = Mp$$

- 什么是齐次坐标？

齐次坐标 (Homogeneous Coordinates)

齐次坐标的本质是使用四维数组来表示三维空间中的点和向量。

Formulation in E^2

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ d & e \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} c \\ f \end{bmatrix}$$

$$p' = M p + t$$

Homogeneous formulation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p' = M p$$

齐次坐标 (Homogeneous Coordinates)

- 由于引入了新的维度，我们使用 4×4 的作用矩阵，同时使用 (x, y, z, w) 来表示一个点或向量：

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

$$p' = M p$$

齐次坐标 (Homogeneous Coordinates)

- 在大部分情况下 $w = 1$, 可以忽略

$$\begin{bmatrix} x' \\ y' \\ z' \\ \color{red}1 \end{bmatrix} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ \color{red}0 & \color{red}0 & \color{red}0 & \color{red}1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ \color{red}1 \end{bmatrix}$$

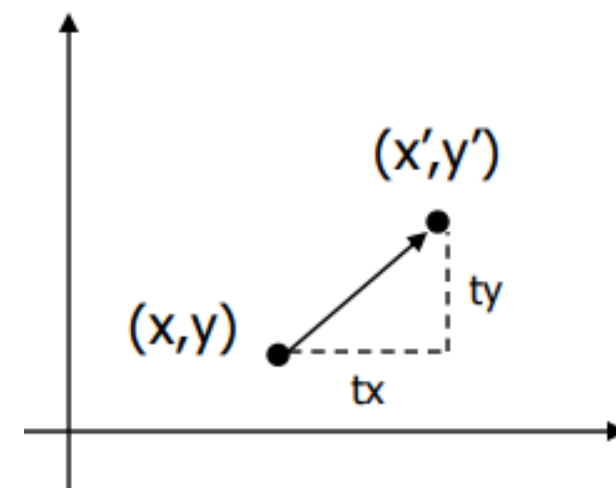
- 当齐次坐标被仿射矩阵作用时, w 不会改变; 而如果被投影矩阵作用, w 会改变。

齐次坐标 (Homogeneous Coordinates)

- 当 w 非零时, 通过将所有四个坐标同时除以 w 以归一化, 例如:
- $(2x, 2y, 2z, 2)$ 等价于 $(x, y, z, 1)$ 。
- 当 w 为零时, 齐次坐标 $(x, y, z, 0)$ 可以理解为沿 (x, y, z) 方向的无穷远点。

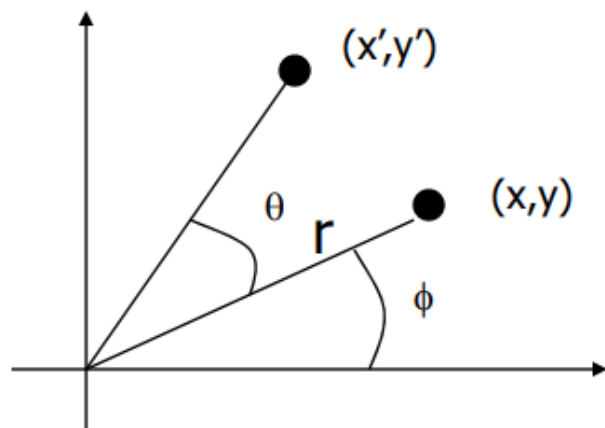
2D平移

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

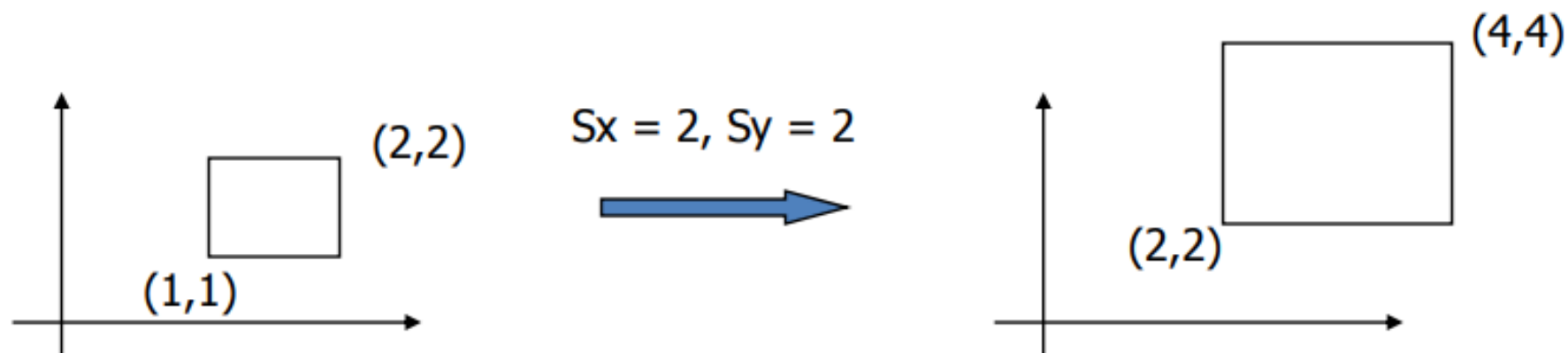


2D旋转

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



2D缩放

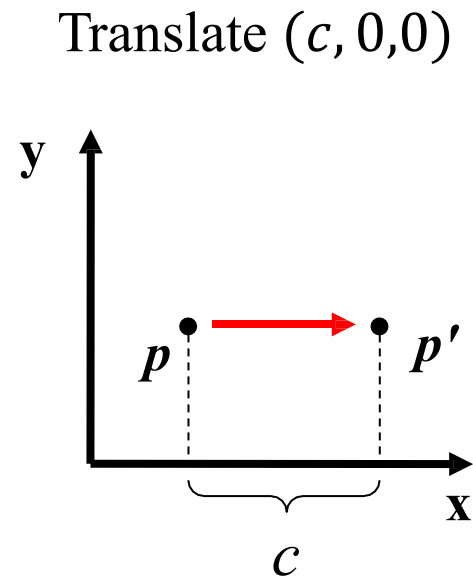


$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

平移变换(t_x, t_y, t_z)

- 可以看到，在齐次坐标的框架下，平移变换可以表示成为简单的矩阵乘法：

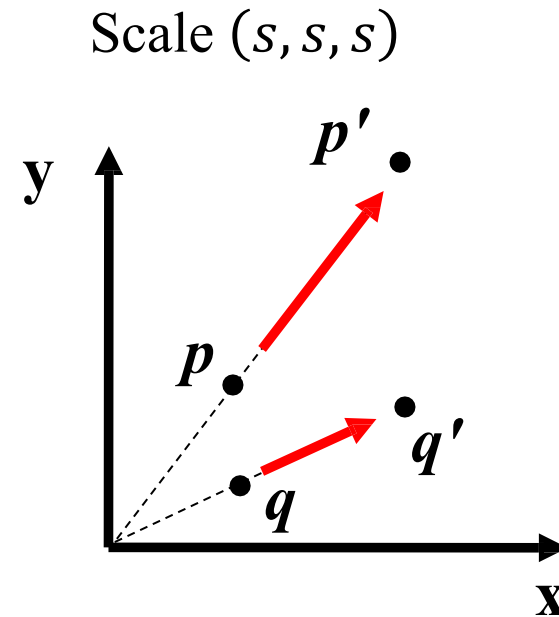
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



缩放变换(s_x, s_y, s_z)

- 以原点为缩放中心的缩放变换可以表示成为：

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



旋转变换

- 三维空间中物体的旋转具有三个独立的自由度。
- 围绕 x 轴的旋转可以表示为：

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$P' = R \cdot P$$

旋转变换

- 三维空间中物体的旋转具有三个独立的自由度。
- 围绕 y 轴的旋转可以表示为：

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$P' = R \cdot P$$

旋转变换

- 三维空间中物体的旋转具有三个独立的自由度。
- 围绕 z 轴的旋转可以表示为：

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$P' = R \cdot P$$

旋转变换

■ 所以

$$\log \mathbf{R} = \theta \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}$$

■ 假设旋转角度 θ 很小时, 代入一阶导近似: $\sin \theta \approx \theta, \cos \theta \approx 1$

■ $R(\omega, \theta)x_i = x_i \cos \theta + (\omega \times x_i) \sin \theta + \omega(\omega^T x_i)(1 - \cos \theta)$

■ $\approx x_i + \theta \omega \times x_i$

Proof

we should proof the equation:

$$\exp(\log(R)) = R$$

let's denote $\begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}$ is A , so

$$\exp(A\theta) = R$$

perform the Taylor series:

$$I + A\theta + \frac{A^2\theta^2}{2!} + \frac{A^3\theta^3}{3!} + \dots = R$$

Proof

Since A is an anti-symmetry matrix, so we have $A^3 = -A$.

Hence, the above equation is $I + (\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} + \dots)A + (\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \dots)A^2$

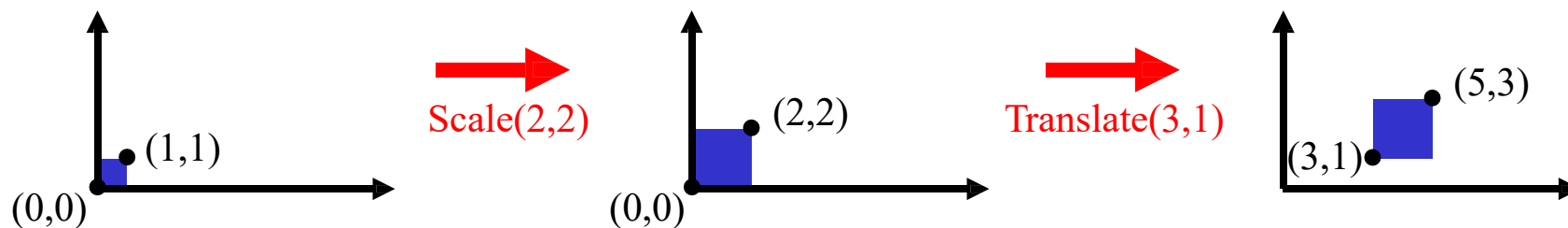
And we know that $\sin(\theta) = \theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} + \dots$, $\cos(\theta) = 1 - (\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \dots)$

Hence, $R = I + \sin(\theta)A + A(1 - \cos(\theta))A$, this is the Rodrigues' formula.

Rodrigues' Formula: https://en.wikipedia.org/wiki/Rodrigues%27_rotation_formula

变换的复合 (combination)

- 一个缩放和平移进行复合的例子：



- 复合等价于矩阵乘法: $p' = T (S p) = TS p$

$$TS = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

变换的复合不满足交换律

- 变换的复合不满足交换律，这是因为矩阵的乘法不满足交换律：

- $TS \neq ST$

- 先缩放再平移：
$$p' = T(Sp) = TS p$$

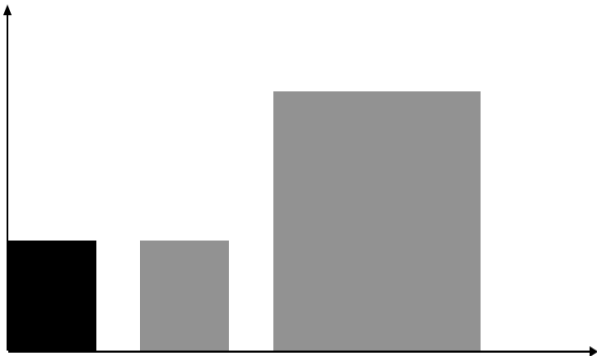
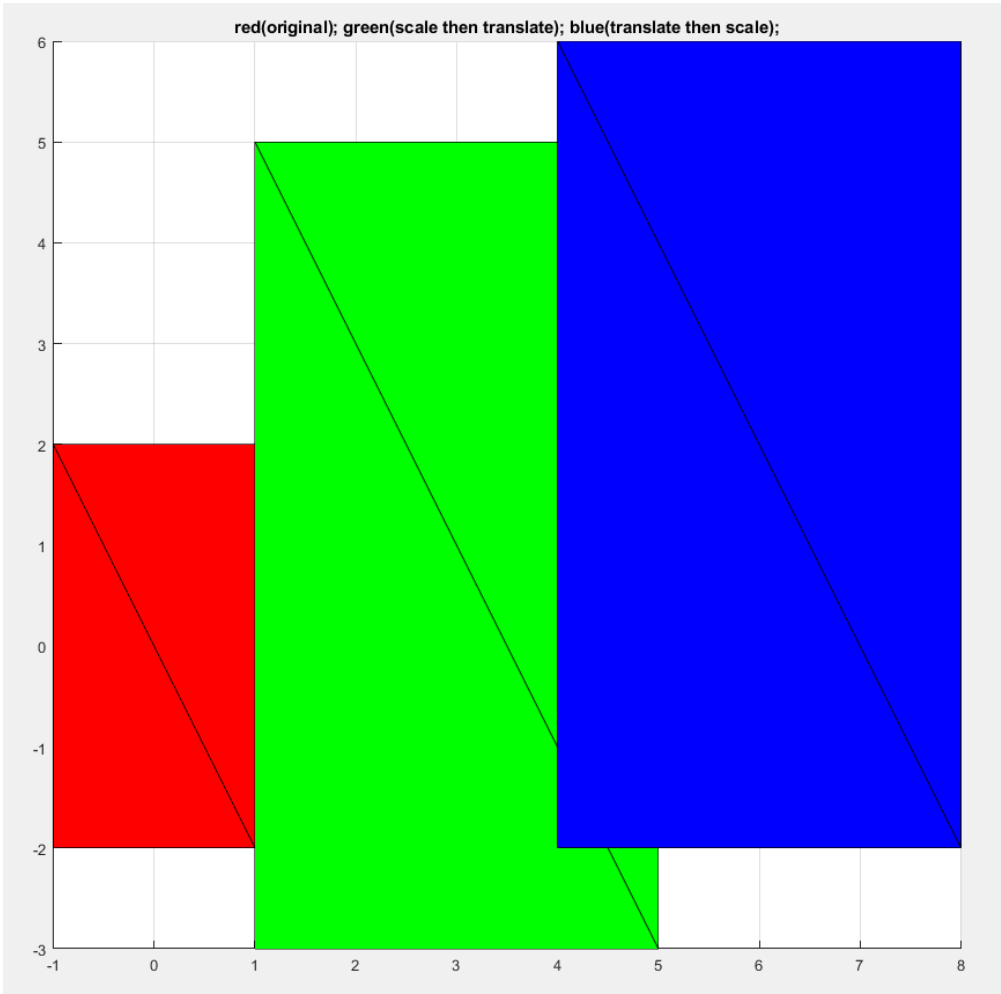
$$TS = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

- 先平移再缩放：
$$p' = S(Tp) = ST p$$

$$ST = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 6 \\ 0 & 2 & 2 \\ 0 & 0 & 1 \end{bmatrix}$$

变换的复合不满足交换律

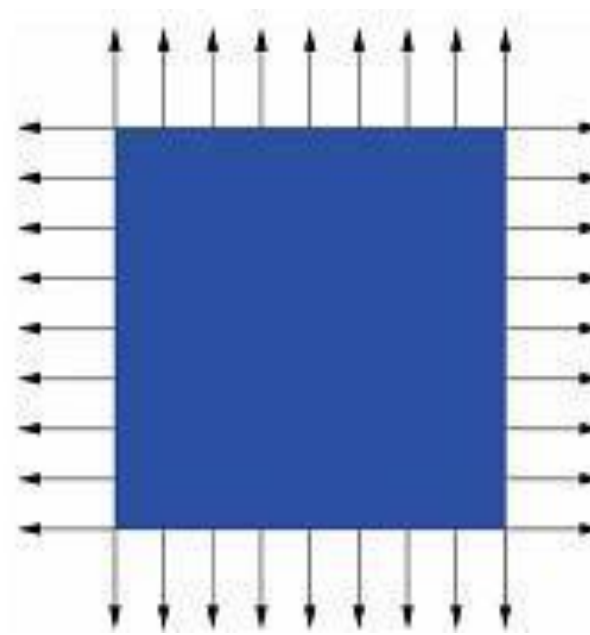
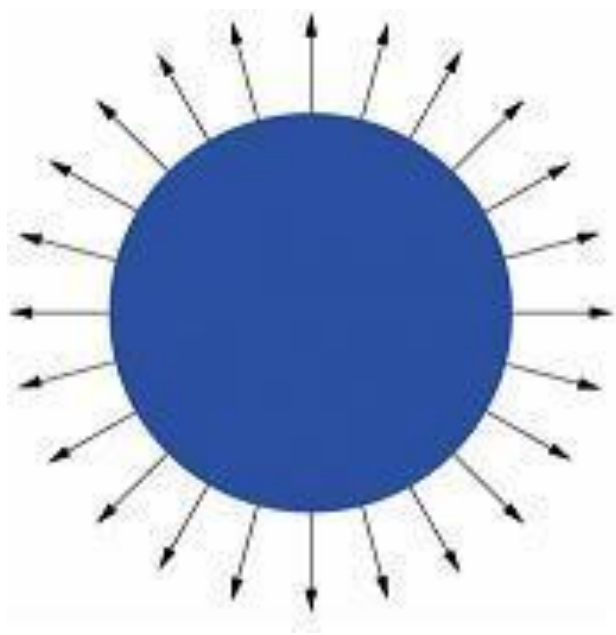
- 变换的复合不满足交换律，这是因为矩阵的乘法不满足交换律：
- $TS \neq ST$



红色：原矩形
绿色：先缩放再平移
蓝色：先平移再缩放

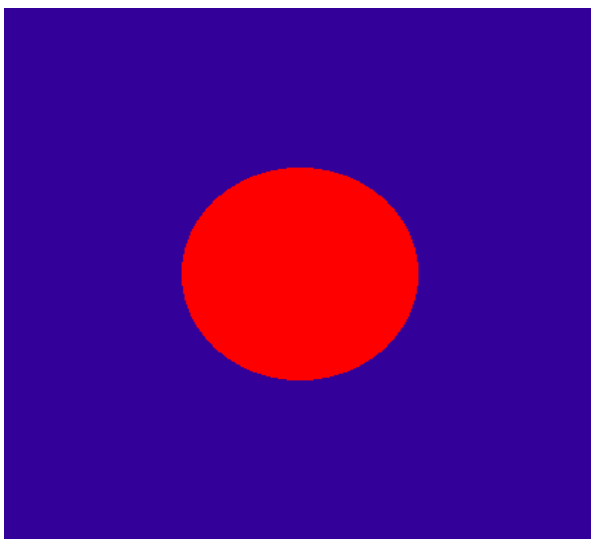
法向量变换

- 曲面的 (单位) 法向量是曲面与曲面正交的 (单位) 向量，它们是曲面最为重要的几何性质之一。

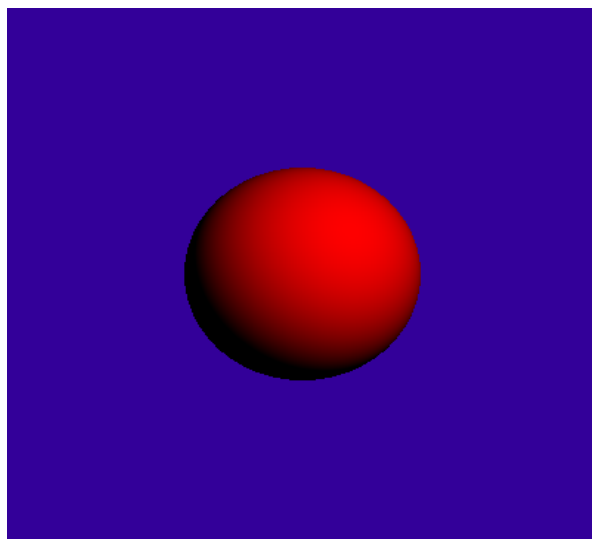


法向量

- 法向量是进行光照处理的必要输入，所有的光照模型都涉及到物体的法向量。只有知道了物体的法向量信息，才能绘制出具有三维立体感的图像。



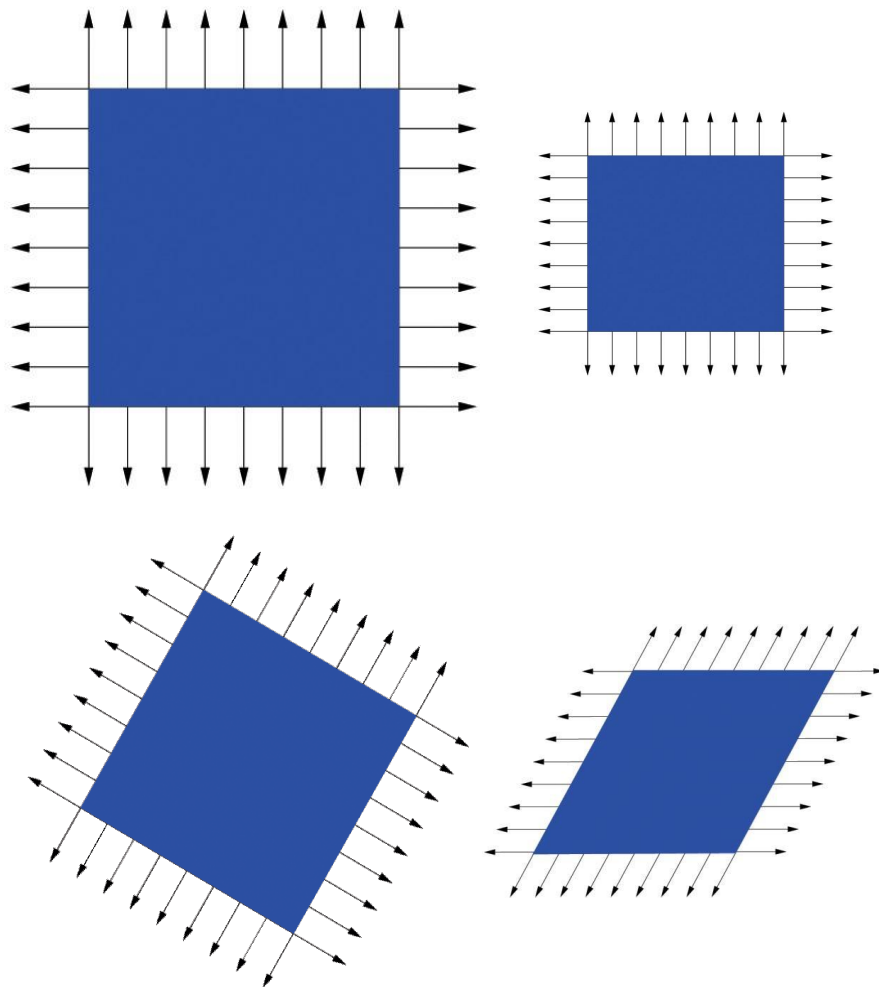
只有物体颜色



漫反射着色

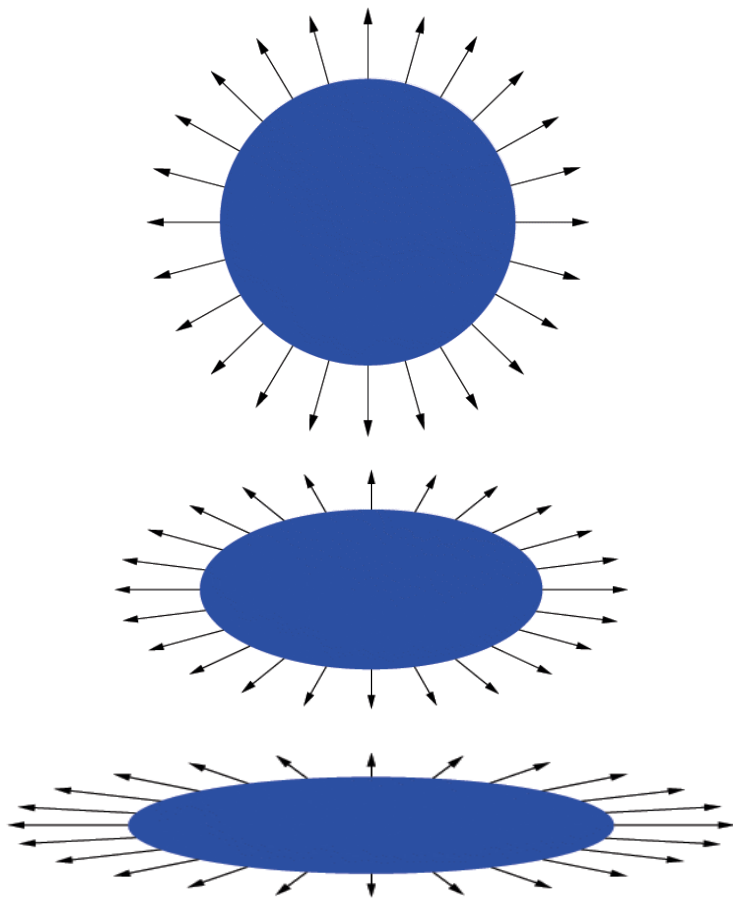
法向量

- 在我们对物体进行变换时，法向量可以类似变换吗？
- 我们发现对于相似变换，法向量可以和物体使用同样的变换方程。然而对于右图中带有错切的仿射变换，同样的方程则不适用。



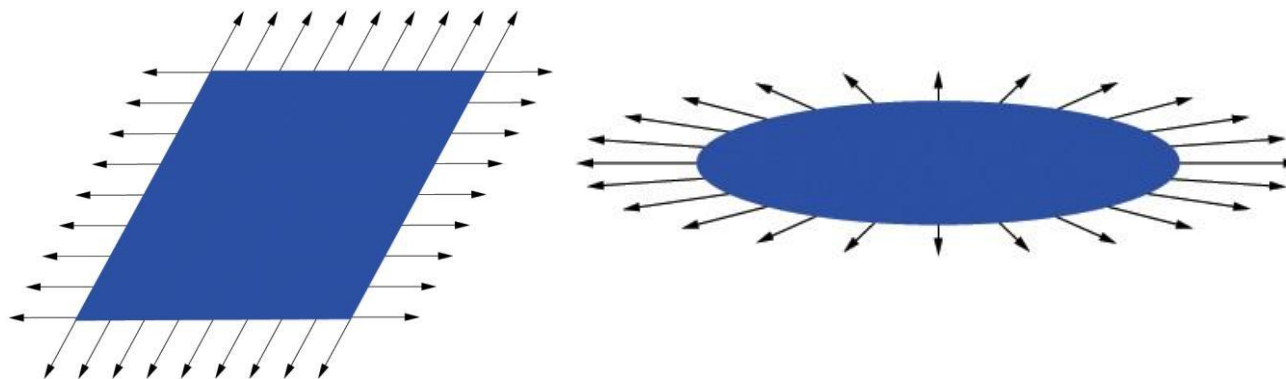
法向量

- 另一个例子（缩放）：
 - ◆ 对右图的球作非均衡的缩放。
 - ◆ 使用与物体同样的变换方程用于变换法向量会出现错误结果：

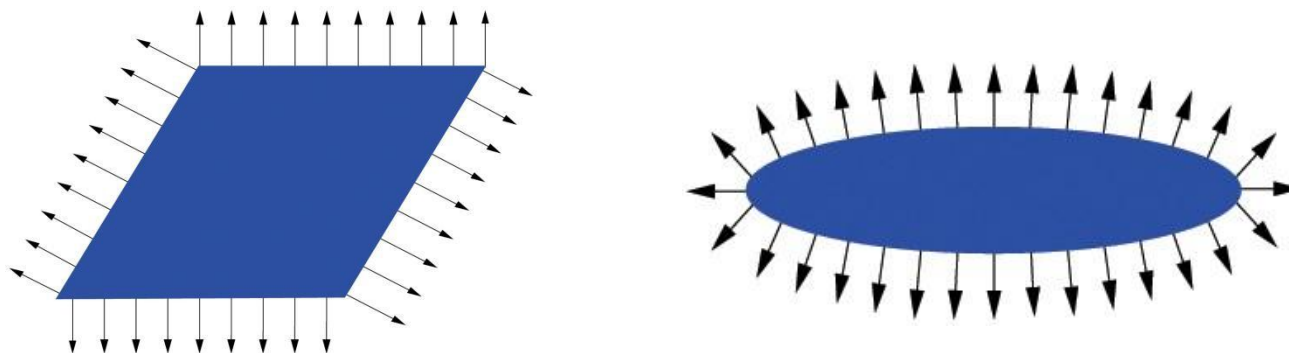


错切和缩放应有的法向变换

错误的法向变换



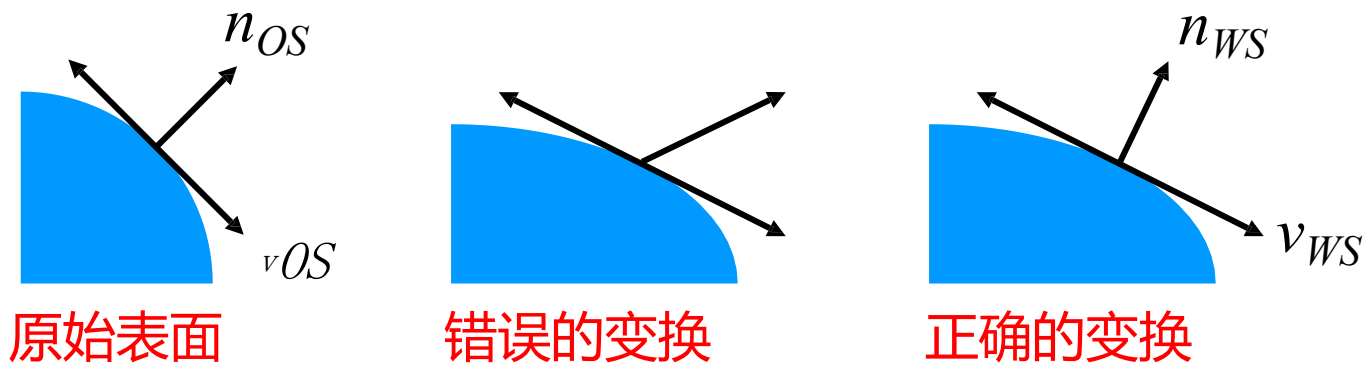
正确的法向变换



如何对法向量进行变换才能确保正确？

如何正确变换法向量？

- 变换切平面 (tangent plane), 再通过切平面计算法向量, 而不是直接计算。



- 切平面上的任一向量 v_{OS} 变换后成为 v_{WS} :

$$v_{WS} = M v_{OS}$$

由切向量计算法向量

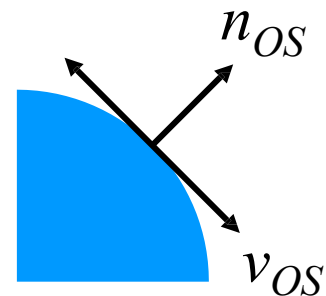
- v_{OS} 和 n_{OS} 垂直:

$$n_{OS}^T v_{OS} = 0$$

$$n_{OS}^T (M^{-1}M) v_{OS} = 0$$

$$(n_{OS}^T M^{-1})(M v_{OS}) = 0$$

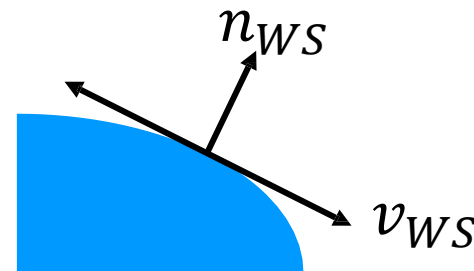
$$(n_{OS}^T M^{-1}) v_{WS} = 0$$



- v_{WS} 和 n_{WS} 垂直:

$$n_{WS}^T = n_{OS}^T = (M^{-1})$$

$$n_{WS} = (M^{-1})^T n_{OS}$$



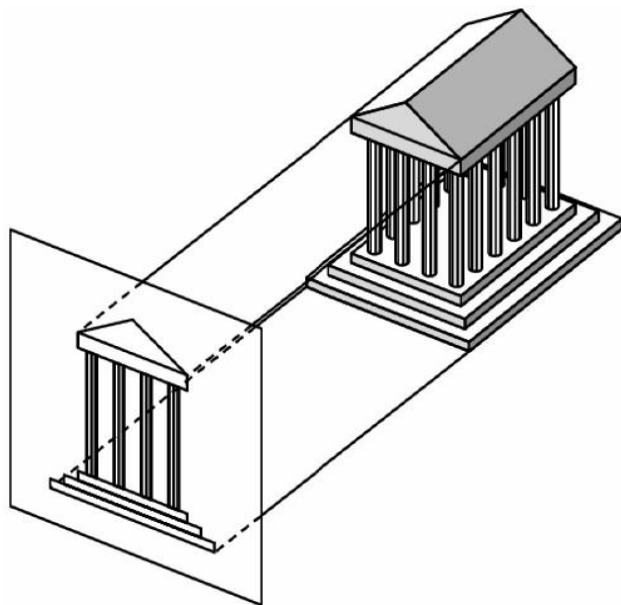
法向量的变换矩阵是原变换矩阵的逆的转置

视点和投影模式

- 我们的眼睛能将三维场景感知为二维图像，大脑则会将二维图像再重构回三维。在计算机图形学中，我们利用投影，模拟眼睛的功能和效果。
- 投影的两个重要概念：
 - ◆ 视点变换：与相机位置及朝向相关
 - ◆ 投影模式：将 3D 变换成为 2D 的变换模式，常用的包括正交投影和透视投影

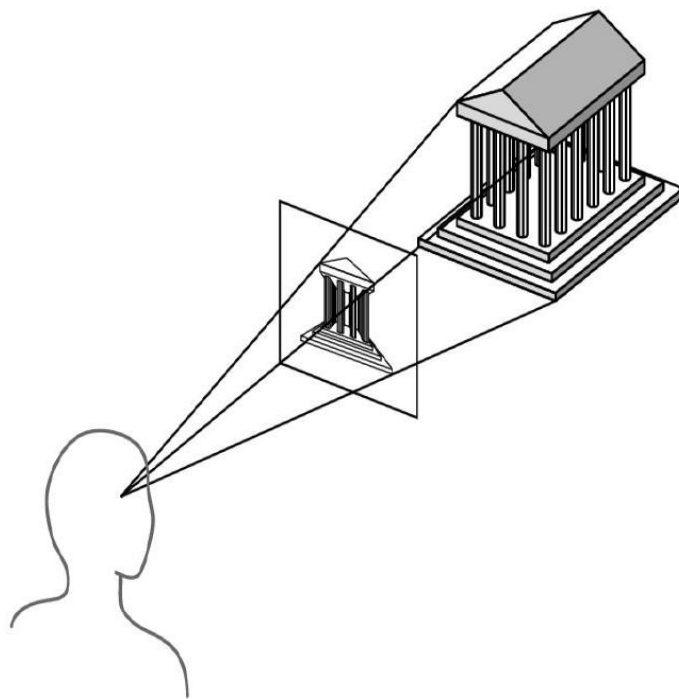
正交投影 (Orthographic Projection)

- 正交投影是视点在无穷远处的投影模式，缺乏立体透视效果。
- 当 xy 平面是投影平面时，正交投影把 (x, y, z) 映射成为 $(x, y, 0)$ 。



透视投影 (Perspective Projection)

- 透视投影是视点在有限距离处的投影模式，具有立体透视效果。
- 当视点位于原点；投影平面为 $z = d$ 时，透视投影把 (x, y, z) 映射成为 $((d/z)x, (d/z)y, d)$ 。



透视投影矩阵

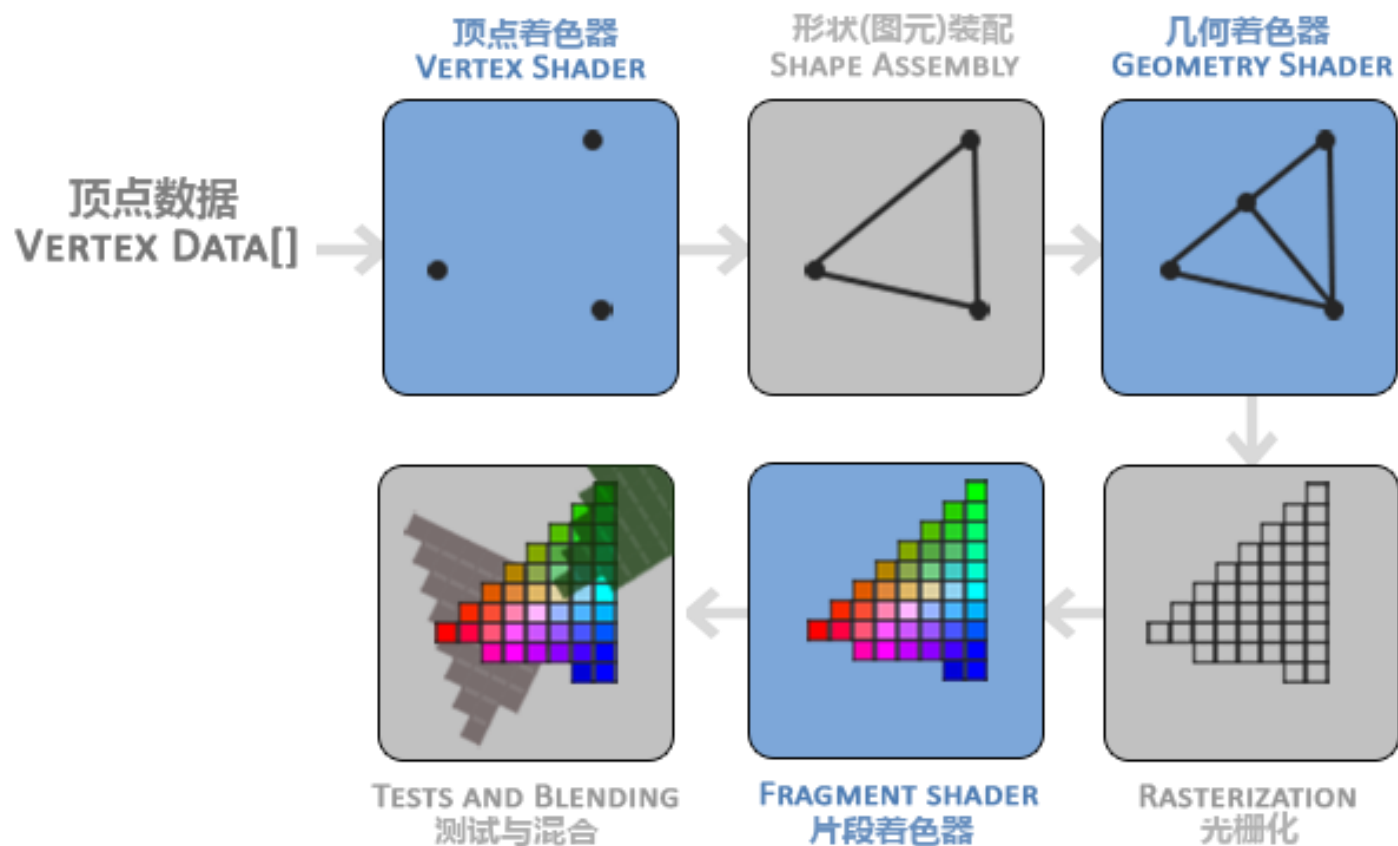
- 使用齐次坐标描述的透视投影变换可以写成矩阵形式：

$$T(x, y, z) = ((d/z)x, (d/z)y, d)$$

$$\begin{bmatrix} dx \\ dy \\ dz \\ z \end{bmatrix} = \begin{bmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & d & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Part 2: 绘制管线

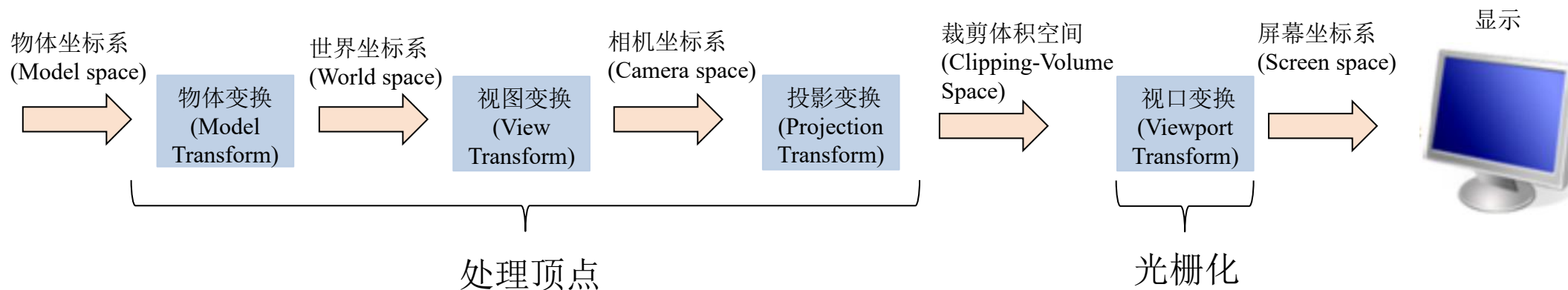
顶点着色器



- 图形渲染管线的第一个部分是顶点着色器(Vertex Shader)，它把一个单独的顶点作为输入。顶点着色器主要的目的是把3D坐标转为另一种3D坐标，同时顶点着色器允许我们对顶点属性进行一些基本处理。

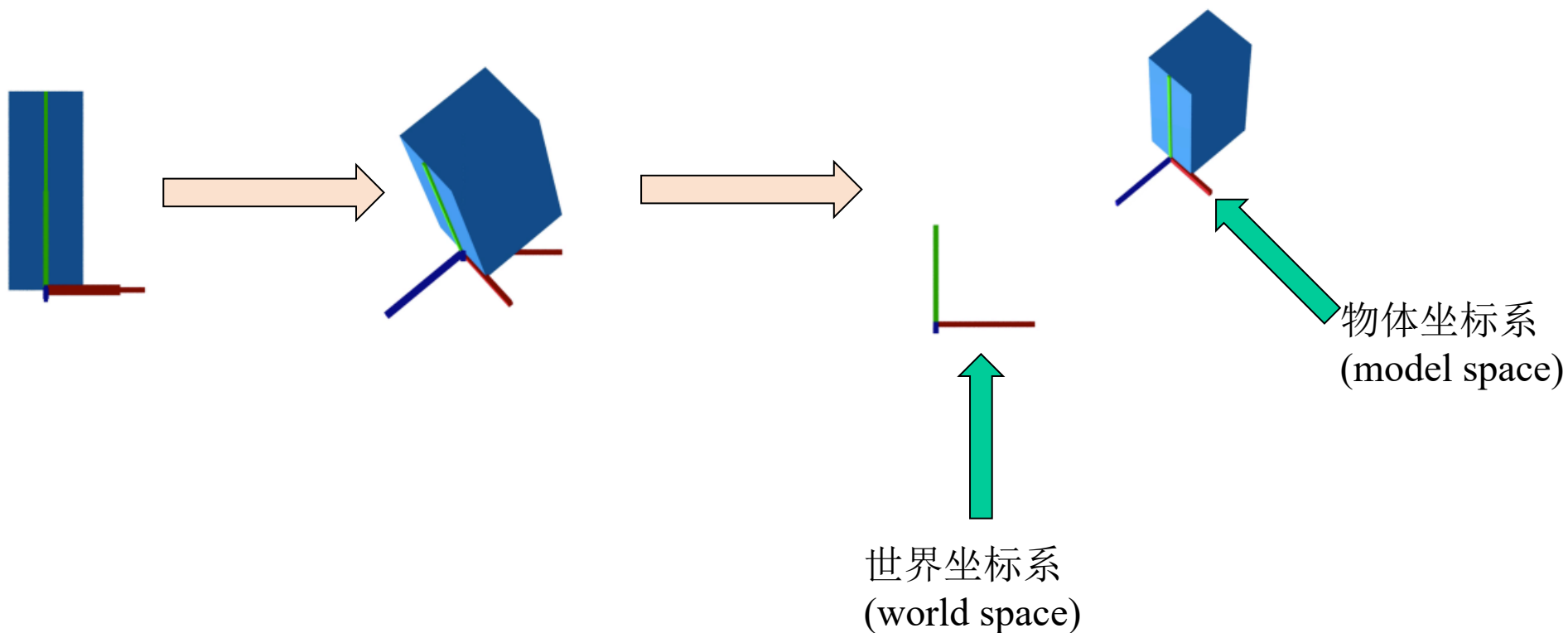
顶点着色器

- 在顶点着色器中，会进行顶点和法相的变换。



物体变换(Model Transform)

- 将物体通过平移、缩放、旋转放置在空间的某个位置。



物体变换(Model Transform)

■ 平移

$$T(d) = \begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

■ 缩放

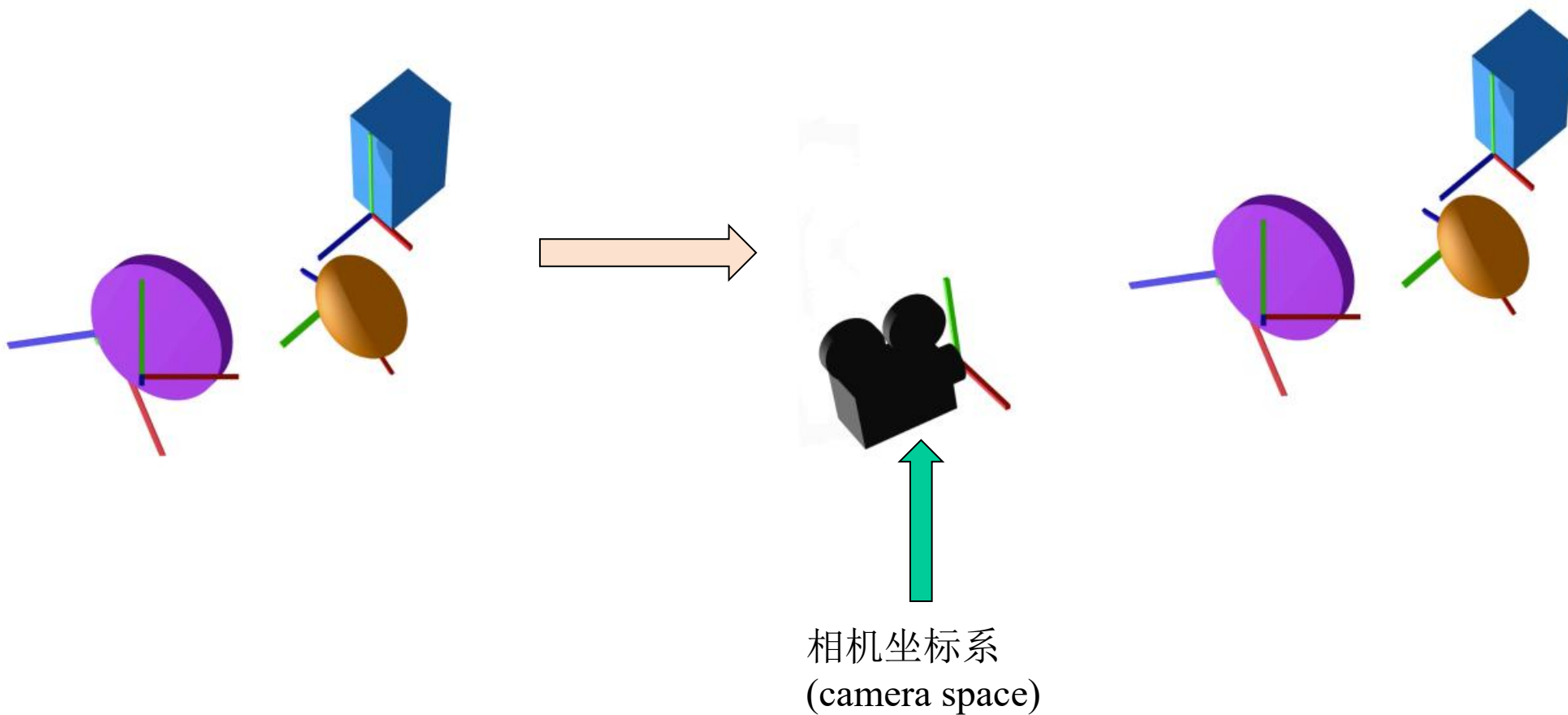
$$S(s) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

■ 旋转

$$R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R_y = \begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

视图变换(View Transform)

- 确定相机位置和朝向。



视图变换(View Transform)

■ 眼睛位置

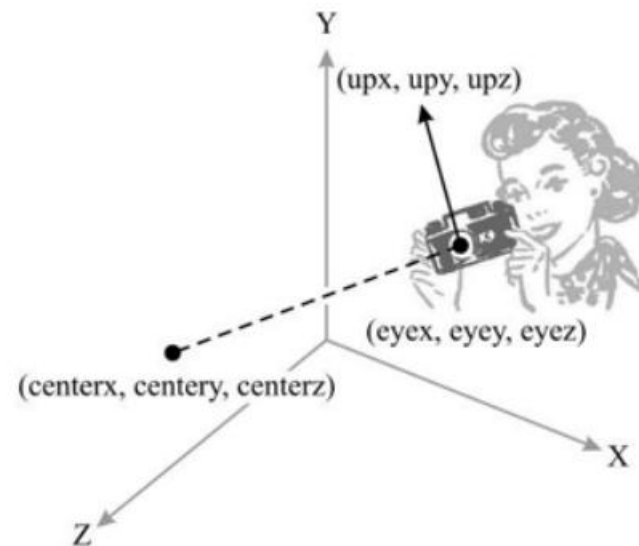
$$eye = \begin{pmatrix} eye_x \\ eye_y \\ eye_z \end{pmatrix}$$

■ 参考位置

$$center = \begin{pmatrix} center_x \\ center_y \\ center_z \end{pmatrix}$$

■ 向上的方向

$$up = \begin{pmatrix} up_x \\ up_y \\ up_z \end{pmatrix}$$



视图变换(View Transform)

■ 计算三个方向的单位向量

$$z^c = \frac{eye - center}{\|eye - center\|}$$

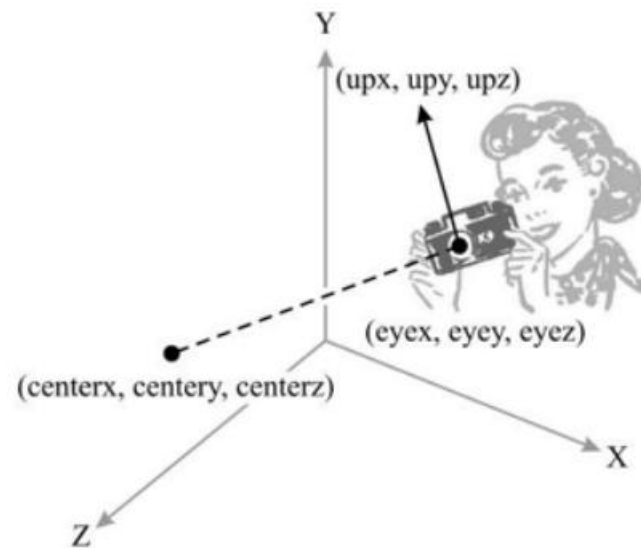
$$x^c = \frac{up \times z^c}{\|up \times z^c\|}$$

$$y^c = z^c \times x^c$$

$$eye = \begin{pmatrix} eye_x \\ eye_y \\ eye_z \end{pmatrix}$$

$$center = \begin{pmatrix} center_x \\ center_y \\ center_z \end{pmatrix}$$

$$up = \begin{pmatrix} up_x \\ up_y \\ up_z \end{pmatrix}$$



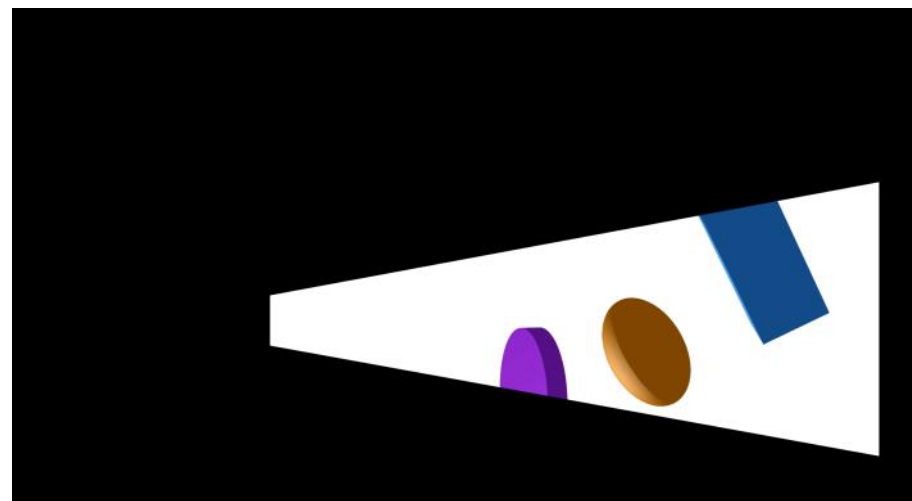
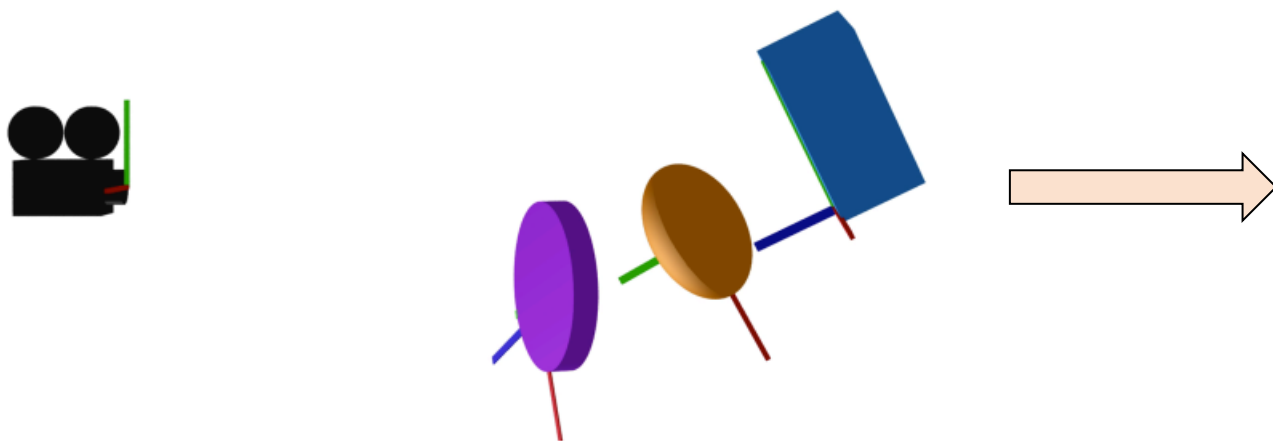
视图变换(View Transform)

- 先平移到眼睛位置，然后旋转。

$$M = R \cdot T(-e) = \begin{pmatrix} x_x^c & x_y^c & x_z^c & 0 \\ y_x^c & y_y^c & y_z^c & 0 \\ z_x^c & z_y^c & z_z^c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -eye_x \\ 0 & 1 & 0 & -eye_y \\ 0 & 0 & 1 & -eye_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$
$$M = R \cdot T(-e) = \begin{pmatrix} x_x^c & x_y^c & x_z^c & -(x_x^c eye_x + x_y^c eye_y + x_z^c eye_z) \\ y_x^c & y_y^c & y_z^c & -(y_x^c eye_x + y_y^c eye_y + y_z^c eye_z) \\ z_x^c & z_y^c & z_z^c & -(z_x^c eye_x + z_y^c eye_y + z_z^c eye_z) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

投影变换(Projection Transform)

- 选择相机镜头（广角、普通或望远镜），调整焦距和变焦系数以设置相机的视野（投影变换）。

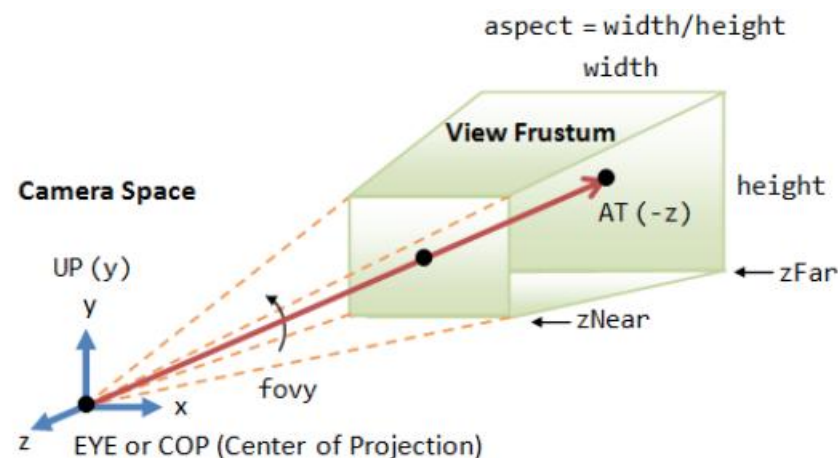


投影变换(Projection Transform)

- 有对称视锥
- $fovy$: 以度为单位的垂直角度
- $aspect$: 宽度/高度的比率
- $zNear$: 近剪裁平面 (相对于相机)
- $zFar$: 远剪裁平面 (相对于相机)

$$f = \cot(fovy / 2)$$

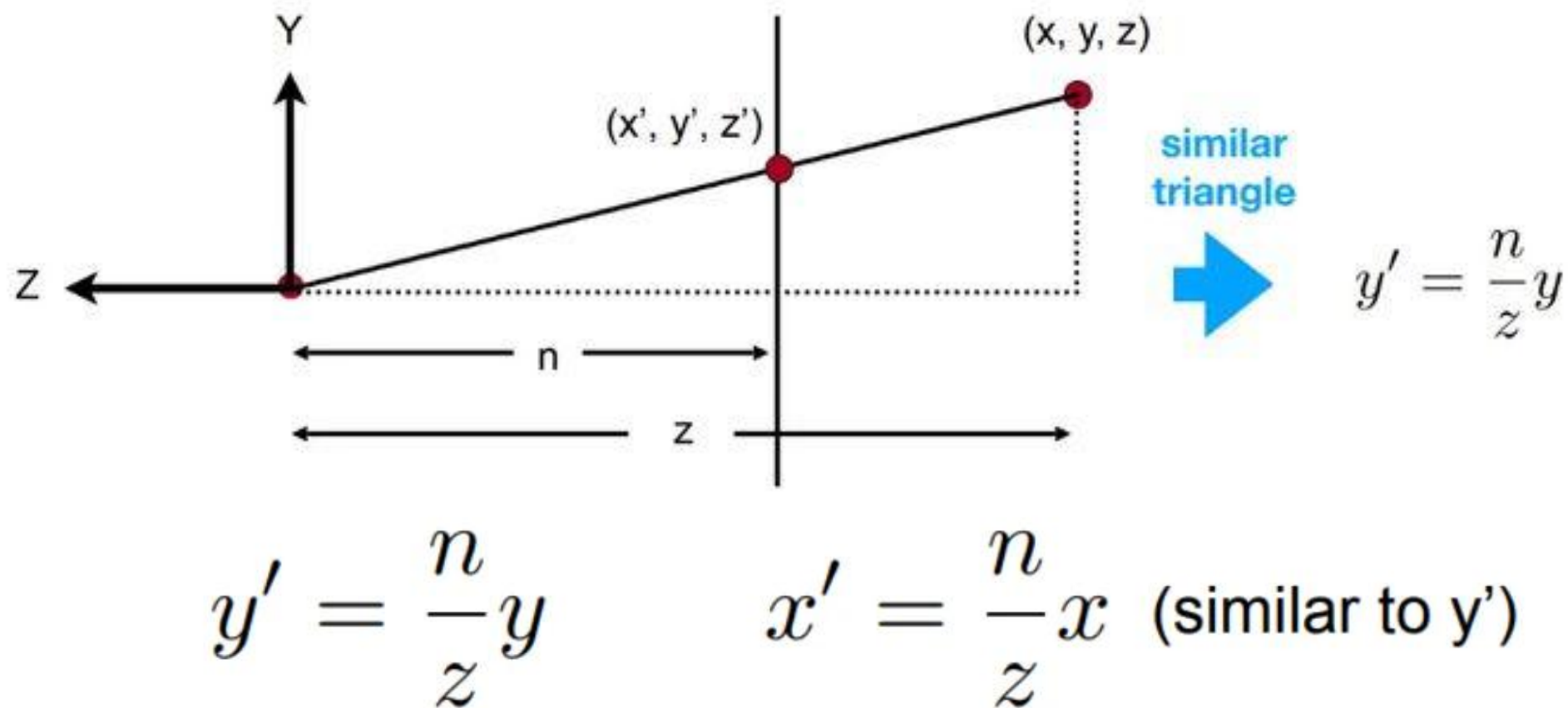
$$M_{proj} = \begin{pmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & -\frac{zFar + zNear}{zFar - zNear} & -\frac{2 \cdot zFar \cdot zNear}{zFar - zNear} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$



Perspective Projection: The camera's view frustum is specified via 4 view parameters: $fovy$, $aspect$, $zNear$ and $zFar$.

透视投影(Perspective Projection)

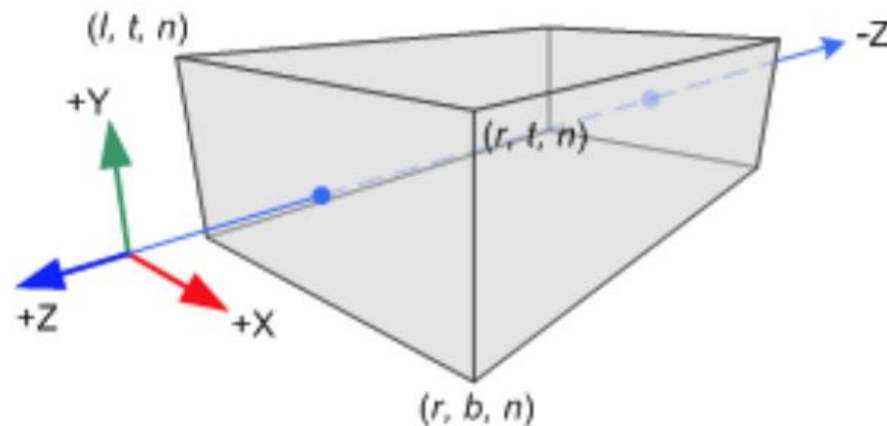
■ 2D情况



正交投影(Orthographic Projection)

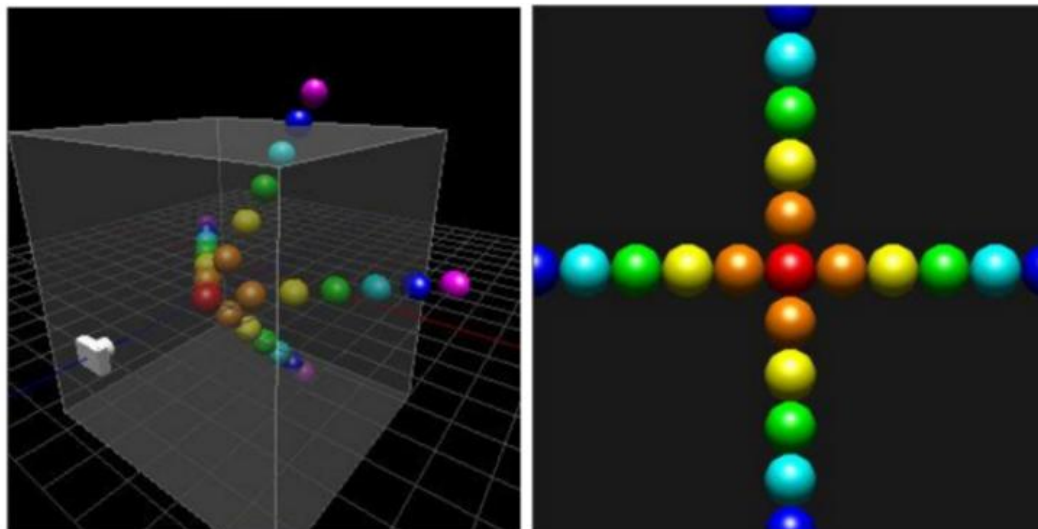
- 方形视锥，物体越远不会变小。
- 左(l), 右(r), 下(b), 上(t)为在近剪裁平面 ($zNear$) 上的各个角顶点坐标。
- 投影矩阵:

$$M_{ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{n-f} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -\frac{r+l}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & -\frac{n+f}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{n-f} & -\frac{n+f}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

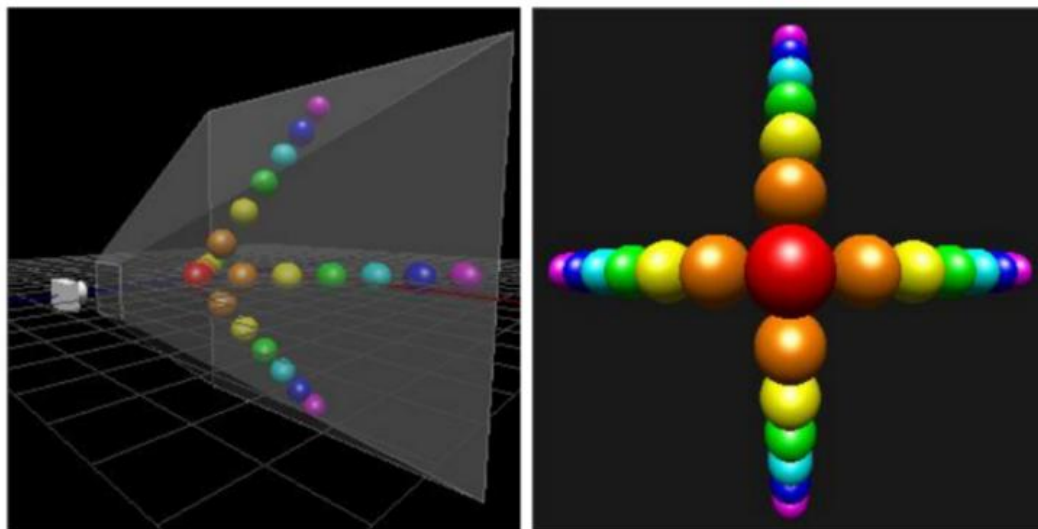


正交投影和透视投影的对比

正交投影

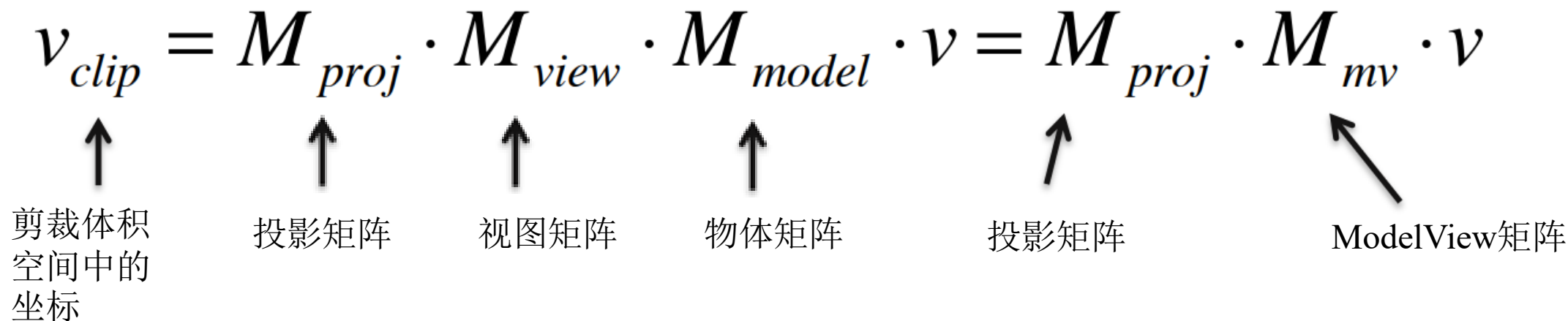


透视投影



Modelview Projection Matrix(MVP Matrix)

- 顾名思义将三个变换矩阵合并起来形成一个4*4的变换矩阵。

$$v_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot v = M_{proj} \cdot M_{mv} \cdot v$$


剪裁体空间中的坐标

投影矩阵

视图矩阵

物体矩阵

投影矩阵

ModelView矩阵

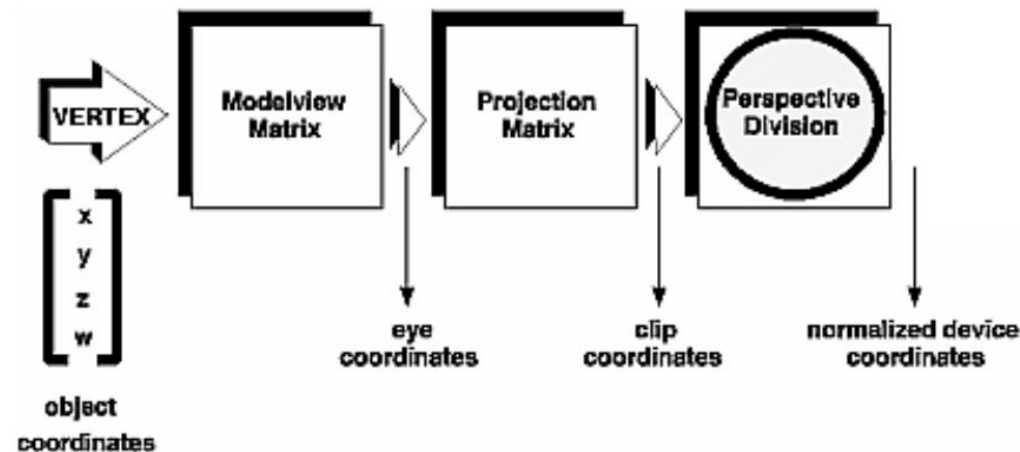
标准化设备坐标 (Normalized Device Coordinates)

- 标准化设备坐标系并未在之前的图片中展示
- 获取标准化设备坐标的方法：

$$v_{clip} = \begin{pmatrix} x_{clip} \\ y_{clip} \\ z_{clip} \\ w_{clip} \end{pmatrix} \longrightarrow v_{NDC} = \begin{pmatrix} x_{clip} / w_{clip} \\ y_{clip} / w_{clip} \\ z_{clip} / w_{clip} \\ 1 \end{pmatrix} \begin{matrix} \in (-1,1) \\ \in (-1,1) \\ \in (-1,1) \end{matrix}$$

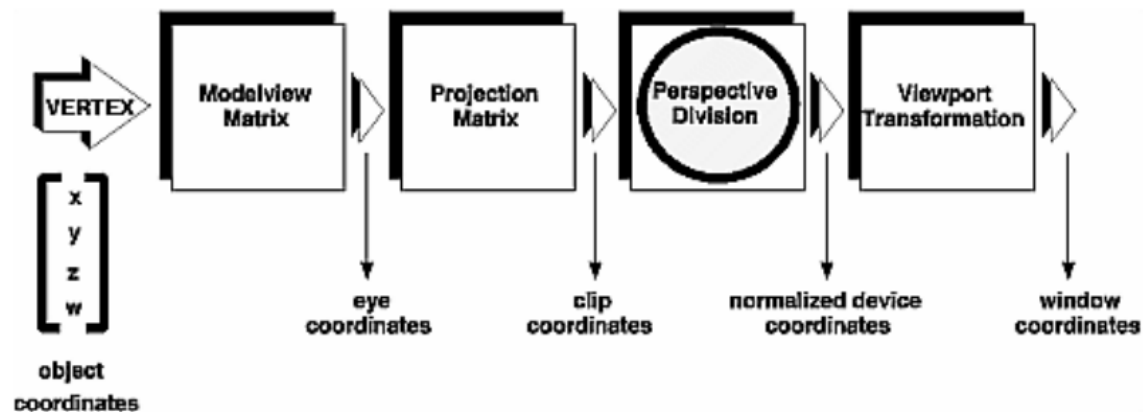
裁剪空间中的坐标

标准化设备坐标系下坐标



视口变换(Viewport Transform)

- 将 (子) 窗口定义为视口 $(x, y, width, height)$
- x, y 为视口矩形的左下角
- $width, height$ 为视口矩形的宽度、高度大小 (以像素为单位)



$$x_{window} = \frac{width}{2}(x_{NDC} + 1) + x$$

$$y_{window} = \frac{height}{2}(y_{NDC} + 1) + y$$

$$z_{window} = \frac{1}{2}z_{NDC} + \frac{1}{2}$$

OpenGL历史

- OpenGL 最初是作为IrisGL的替代品被创建的，IrisGL是Silicon Graphics工作站上的专有图形API。Mark Segal和Kurt Akeley 在Silicon Graphics公司工作期间撰写了OpenGL 1.0编程规范，试图将经常使用的图形API的定义规范化，从而使跨平台实现成为可能。
- OpenGL API的1.0版本并非完美，一个显著缺点是其缺少纹理对象，而IrisGL对各种对象（包括材料、灯光、纹理和纹理环境）都有定义和绑定阶段。
- OpenGL避开了这些对象，转而支持增量状态更改，也即状态机，认为这些对象的更改可以封装在显示列表中。

OpenGL历史

■ 修订历史:

- ◆ OpenGL 1.1 将glBindTexture添加到核心 API 中。
- ◆ OpenGL 2.0 包含了 OpenGL 阴影语言（也称为 GLSL）的重要添加，这是一种类似于C的语言，可以用该语言对管道的转换和片段阴影阶段进行编程。
- ◆ OpenGL 3.0 增加了弃用的概念：在以后的版本中将某些功能标记为可删除。
- ◆ OpenGL 3.1 删除了大多数弃用的功能。
- ◆ OpenGL 3.2 创建了可调和兼容性 OpenGL 上下文的概念。
- ◆ 迄今发布的OpenGL官方版本为 1.0、1.1、1.2、1.2.1、1.3、1.4、1.5、2.0、2.1、3.0、3.1、3.2、3.3、4.0、4.1、4.2、4.3、4.4、4.5、4.6。

OpenGL标准撰写人

■ Kurt Akeley:

- ◆ 1982年斯坦福大学硕士毕业后在SGI工作20年，期间制定了OpenGL标准，并升职为CTO。
- ◆ 2001-2004斯坦福大学博士，师从图灵奖得主Pat Hanrahan
- ◆ 斯坦福大学计算机图形实验室咨询助理教授
- ◆ 1995获得SIGGRAPH成就奖
- ◆ 2005年入选美国工程院
- ◆ 2010年担任Lytro, Inc. CTO



<https://www.siggraph.org/member-profile/kurt-akeley/>
https://en.wikipedia.org/wiki/Kurt_Akeley

谢谢！ 欢迎交流！

高林

gaolin@ict.ac.cn

<http://www.geometrylearning.com>